



Workshop Raspberry Pi Pico

Dreiteiliger Workshop zu den Grundlagen
des Raspberry Pi Pico

02.06.2026 / 09.06.2026 / 16.06.2026

jeweils 14:00 Uhr bis 17:30 Uhr

Campus Rüsselsheim, Raum A-U31

Anmeldung über Stud.IP



https://studip.hs-rm.de/dispatch.php/course/details?sem_id=2e2826130a46bcd2c4379297432876ff&again=yes



<https://umfragen.hs-rm.de/evasys/online.php?p=StuBe26>

STUDIUM 10/10

ODER NOCH LUFT NACH OBEN?

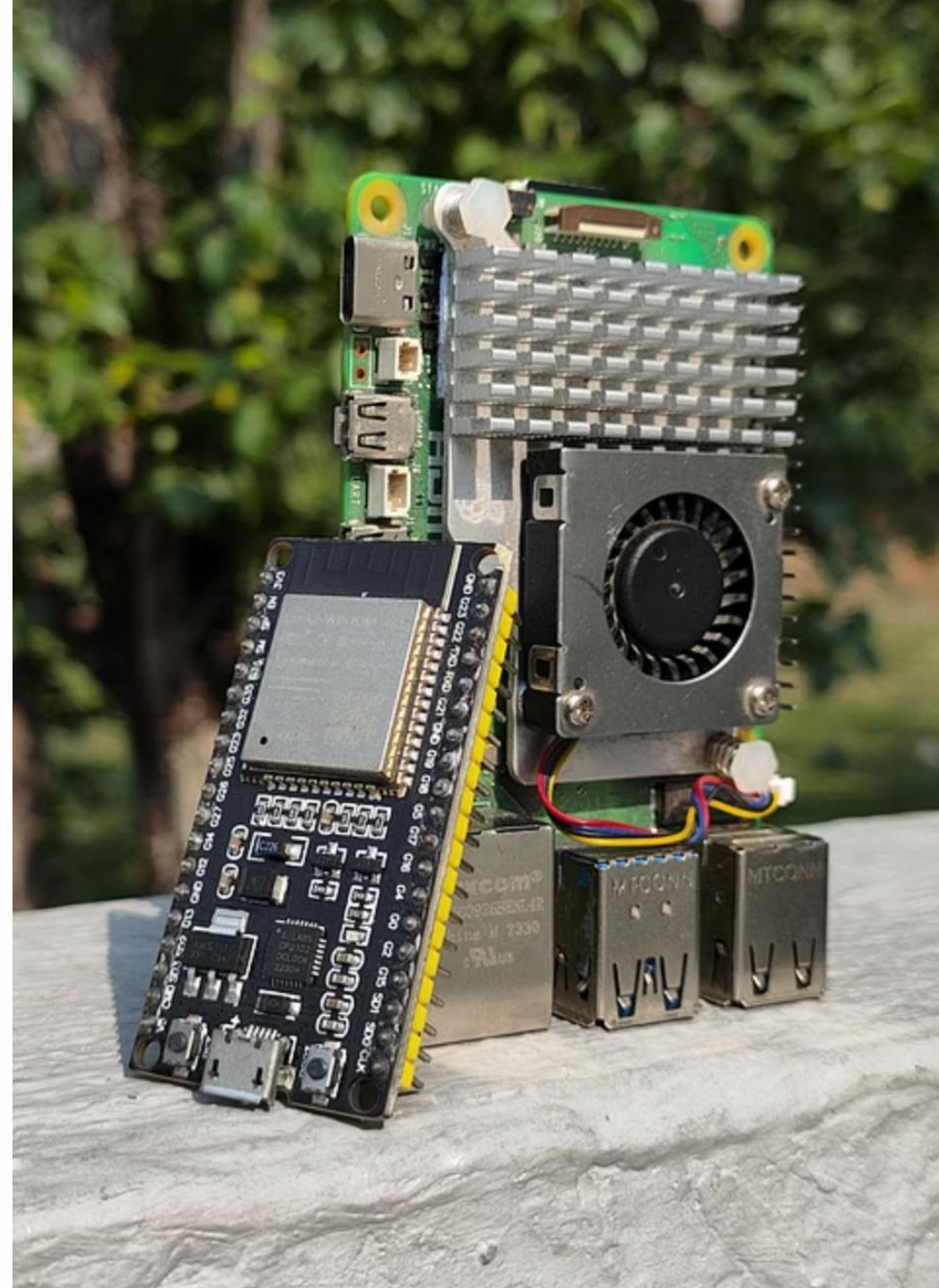
hsrm*

Verbessere dein Studium:
Nimm an der Befragung teil!



Einplatinenmaschinen im IoT: ESP32, Raspberry Pi und verwandte Plattformen

Aufbau, Fähigkeiten, Grenzen und technische Trade-offs in IoT-Systemen



Einplatinenmaschinen im Kontext des Internet of Things

Leitfrage der Vorlesung

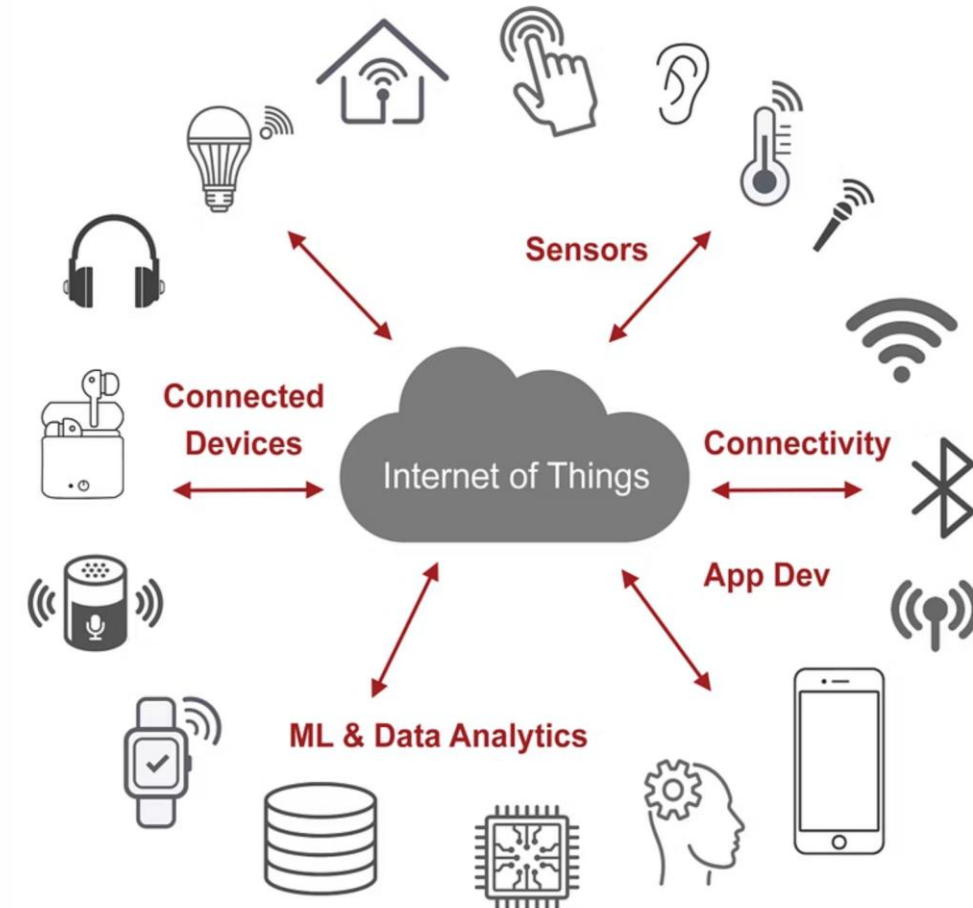
Welche Plattform ist für welches IoT-Szenario technisch sinnvoll?

Begriffe sauber trennen: **Microcontroller Board, SoC, SBC** und **Edge Device** bezeichnen unterschiedliche Abstraktionsebenen.

Einordnung

Einplatinenmaschinen und Mikrocontrollerboards bilden die physische Rechenbasis vieler IoT-Systeme. Die Plattformen reichen von sehr energieeffizienten Mikrocontroller-Systemen wie dem **ESP32** bis zu leistungsfähigeren Single-Board-Computern wie dem **Raspberry Pi**.

Die Geräte unterscheiden sich grundlegend in Architektur, Betriebssystemfähigkeit, Echtzeitverhalten, Energiebedarf und Integrationsaufwand.



Einordnung: Was ist eine Einplatinenmaschine?

Eine Einplatinenmaschine integriert Recheneinheit, Speicher, Stromversorgungsschnittstellen und I/O auf einer einzigen Leiterplatte. Im IoT treten zwei Klassen besonders häufig auf.

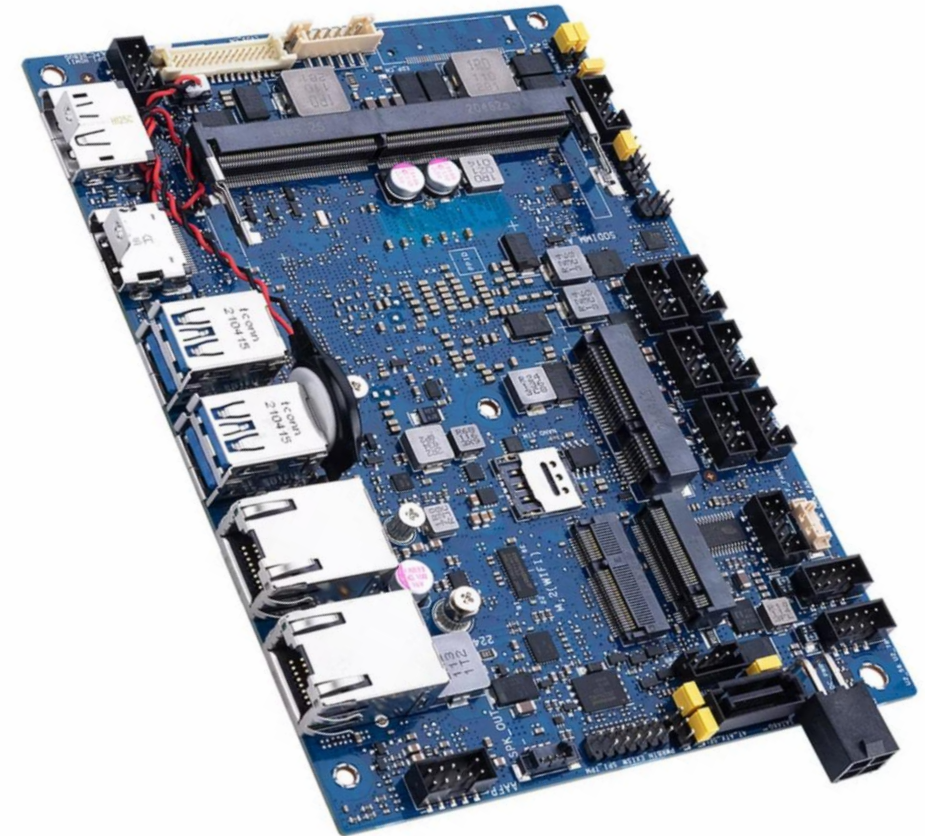
Mikrocontroller-basierte Boards

Integrieren Prozessor, Speicher und Peripherie eng auf einem Chip. Ausgelegt auf deterministische, energiearme Steuerungsaufgaben. **Kein klassischer Betriebssystembetrieb**; direkte Hardwarenähe steht im Vordergrund.

Single-Board-Computer (SBC)

Bündeln CPU, RAM, Grafik- und I/O-Funktionen auf einer Platine. Eignen sich für komplexere Software-Stacks **bis hin zu Linux**-basierten Edge-Anwendungen. Höherer Energiebedarf, aber deutlich mehr Rechenleistung und Flexibilität.

i Nicht jede Einplatinenmaschine ist ein vollwertiger Computer; viele sind primär eingebettete Steuerungssysteme.



Mikrocontrollerboard vs. Single-Board-Computer

MCU-BOARD

Typisch: ESP32, STM32, nRF52

- Kein klassischer Desktop-Betriebssystembetrieb
- Sehr geringer Energiebedarf – geeignet für Batteriebetrieb
- Enge GPIO-Nähe und gutes Echtzeitverhalten
- Eher für Sensorik, Aktorik, Funk und Embedded Control
- RTOS oder Bare-Metal-Betrieb typisch

SBC

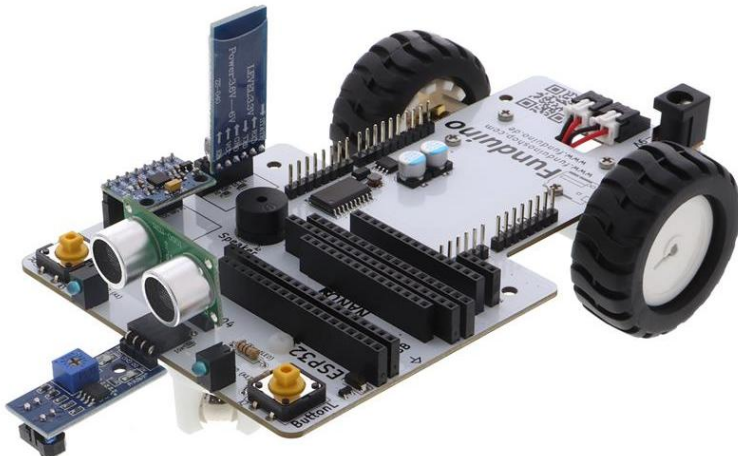
Typisch: Raspberry Pi, BeagleBone, ODROID

- Linux/Unix-artige Systeme möglich
- Höhere Rechenleistung und RAM-Kapazität
- Höherer Energiebedarf, komplexerer Software-Stack
- Eher für Edge Analytics, Gateways, lokale Dienste
- Geeignet für Kameraverarbeitung und containernahe Workflows

MCU – Microcontroller Unit · SBC – Single-Board Computer · GPIO – General Purpose Input/Output · RTOS – Real-Time Operating System · ESP32 – Espressif 32-bit SoC · STM32 – STMicroelectronics 32-bit MCU · nRF52 – Nordic Semiconductor MCU



Warum sind diese Plattformen für IoT zentral?



Nähe zur physischen Welt

Direkter Zugriff auf Sensoren, Busse und Aktoren über GPIO, I2C, SPI und UART.

Vernetzbarkeit

WLAN, Bluetooth, Ethernet und Zusatzmodule ermöglichen Edge-to-Cloud-Kommunikation.

Verteilbarkeit

Kleine Bauform, niedrige Kosten und modulare Erweiterbarkeit begünstigen breite Feldinstallation.

Edge-Fähigkeit

Vorverarbeitung, Filterung und lokale Regelung reduzieren Latenz, Bandbreite und Cloud-Abhängigkeit.



IoT-Hardware ist nicht nur „klein“, sondern ein Kompromissraum aus Rechenleistung, Energie, Konnektivität und Systemkomplexität.

GPIO – General Purpose Input/Output · I2C – Inter-Integrated Circuit · SPI – Serial Peripheral Interface · UART – Universal Asynchronous Receiver-Transmitter · WLAN – Wireless Local Area Network · BT – Bluetooth · IoT – Internet of Things

Grundaufbau einer IoT-Einplatinenmaschine

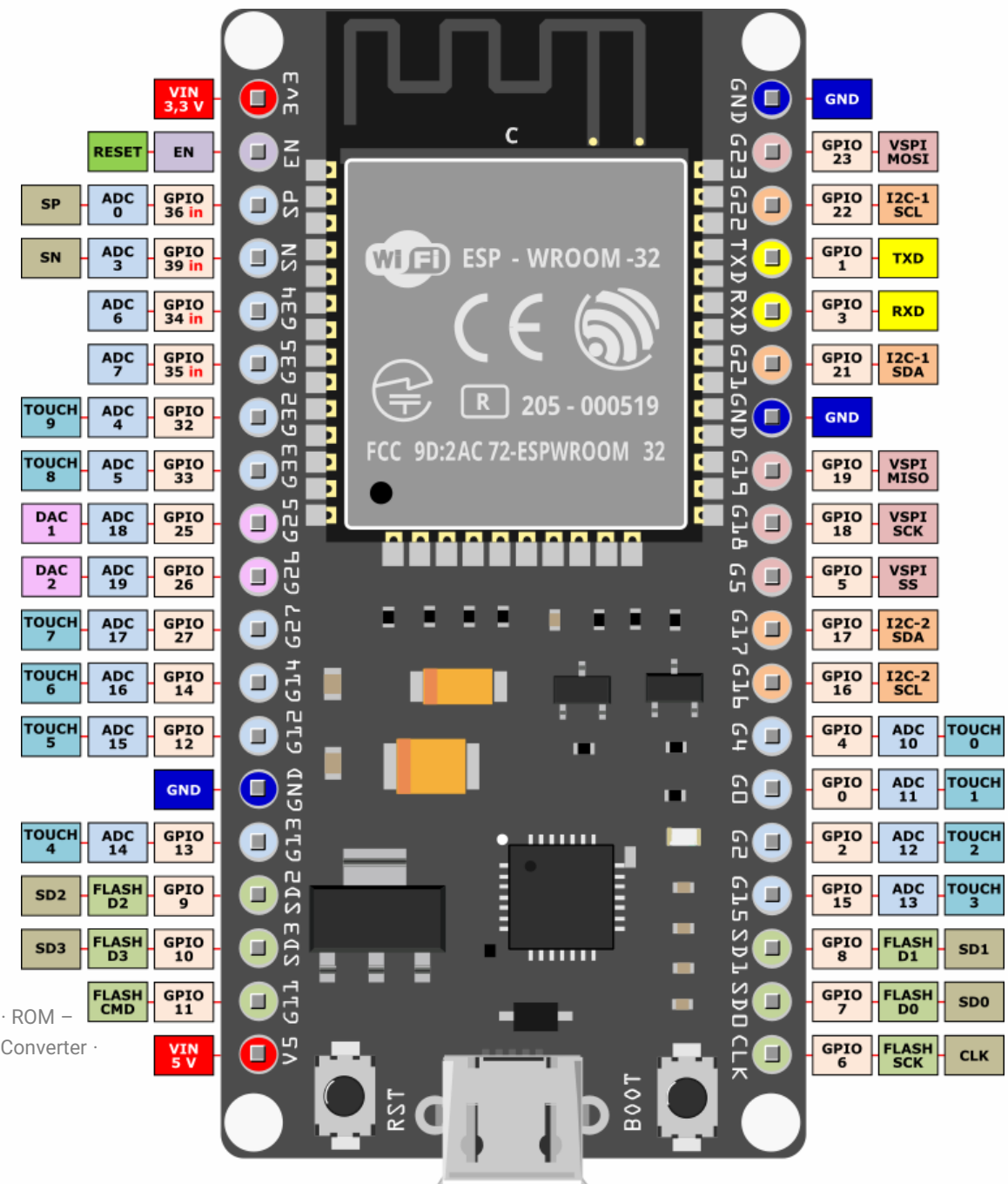
Typische Bausteine

- **SoC bzw. MCU/CPU** – zentrales Rechenwerk
- **Flüchtiger Speicher (RAM)** – Laufzeitdaten und Stack
- **Nichtflüchtiger Speicher (Flash/ROM)** – Firmware und Konfiguration
- **Funk- oder Netzwerkschnittstellen** – WLAN, BT, Ethernet
- **I/O-Peripherie** – GPIO, ADC, DAC, Timer, Busse
- **Takt- und Power-Management** – Taktgeneratoren, Sleep-Modi
- **Debug-/Programmierschnittstelle** – JTAG, SWD, UART-Bootloader

Integrationsprinzip

Im IoT ist besonders relevant, wie eng Rechenwerk, Speicher, Funk und Peripherie auf einem Chip oder einer Platine zusammengefasst sind. Je stärker das System integriert ist, desto geringer sind oft Energiebedarf und Stückkosten – aber desto enger werden auch Speicher- und Funktionsgrenzen.

SoC – System on a Chip · MCU – Microcontroller Unit · CPU – Central Processing Unit · RAM – Random Access Memory · ROM – Read-Only Memory · GPIO – General Purpose Input/Output · ADC – Analog-to-Digital Converter · DAC – Digital-to-Analog Converter · JTAG – Joint Test Action Group · SWD – Serial Wire Debug · UART – Universal Asynchronous Receiver-Transmitter



Architektur des ESP32

Kern und Funk

- **Dual-Core Xtensa LX6** bis 240 MHz
- Integriertes **WLAN 802.11 b/g/n**
- **Bluetooth BR/EDR** und **BLE**
- Interne ROM-, SRAM- und RTC-Speicherbereiche

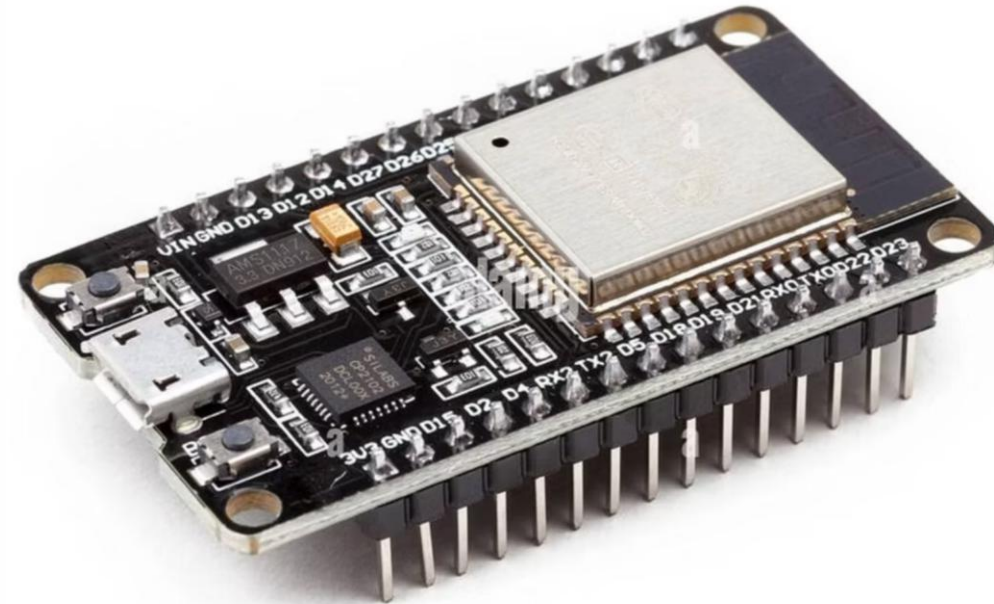
Sicherheitsfunktionen

- Secure Boot
- Flash Encryption
- Kryptographische Hardwarebeschleuniger (AES, SHA, RSA)

Peripherie

- **ADC, DAC** und kapazitive Touch-Sensoren
- Flexible **GPIO**-Konfiguration
- Busse: **SPI, I2C, I2S, UART, SDIO**
- **PWM**-Controller und Hardware-Timer

Der ESP32 ist ein vollständiger Mikrocontroller-SoC mit integrierter Funkkommunikation – kein separater Funk-Transceiver erforderlich. Quelle: Espressif ESP32 Technical Reference Manual.



ESP32 – Espressif Systems 32-bit SoC · SoC – System on a Chip · WLAN – Wireless Local Area Network · BLE – Bluetooth Low Energy · BR/EDR – Basic Rate/Enhanced Data Rate · ROM – Read-Only Memory · SRAM – Static Random Access Memory · RTC – Real-Time Clock · ADC – Analog-to-Digital Converter · DAC – Digital-to-Analog Converter · GPIO – General Purpose Input/Output · SPI – Serial Peripheral Interface · I2C – Inter-Integrated Circuit · I2S – Inter-IC Sound · UART – Universal Asynchronous Receiver-Transmitter · SDIO – Secure Digital Input Output · PWM – Pulse Width Modulation · AES – Advanced Encryption Standard · SHA – Secure Hash Algorithm · RSA – Rivest–Shamir–Adleman

ESP32: Was kann das Gerät gut?

Der ESP32 ist stark bei energieeffizienter, netzwerkfähiger Embedded-Steuerung. Die enge Kopplung von Funk, GPIO und Peripherie auf einem einzigen Chip macht ihn besonders attraktiv für IoT-Knoten mit begrenzter Energieversorgung.



Vernetzte Sensorik

Direkte WLAN/BLE-Konnektivität ohne Zusatzmodule. Geeignet für Smart-Home-Knoten und Telemetrie.



Batterienahe Systeme

Tiefe Sleep-Modi erlauben Laufzeiten von Monaten mit kleinen Akkus. Ideal für Wearables und Umweltsensoren.



Embedded Control

Lokale Steuerlogik, einfache Edge-Entscheidungen und Schaltaufgaben mit geringem Software-Overhead.

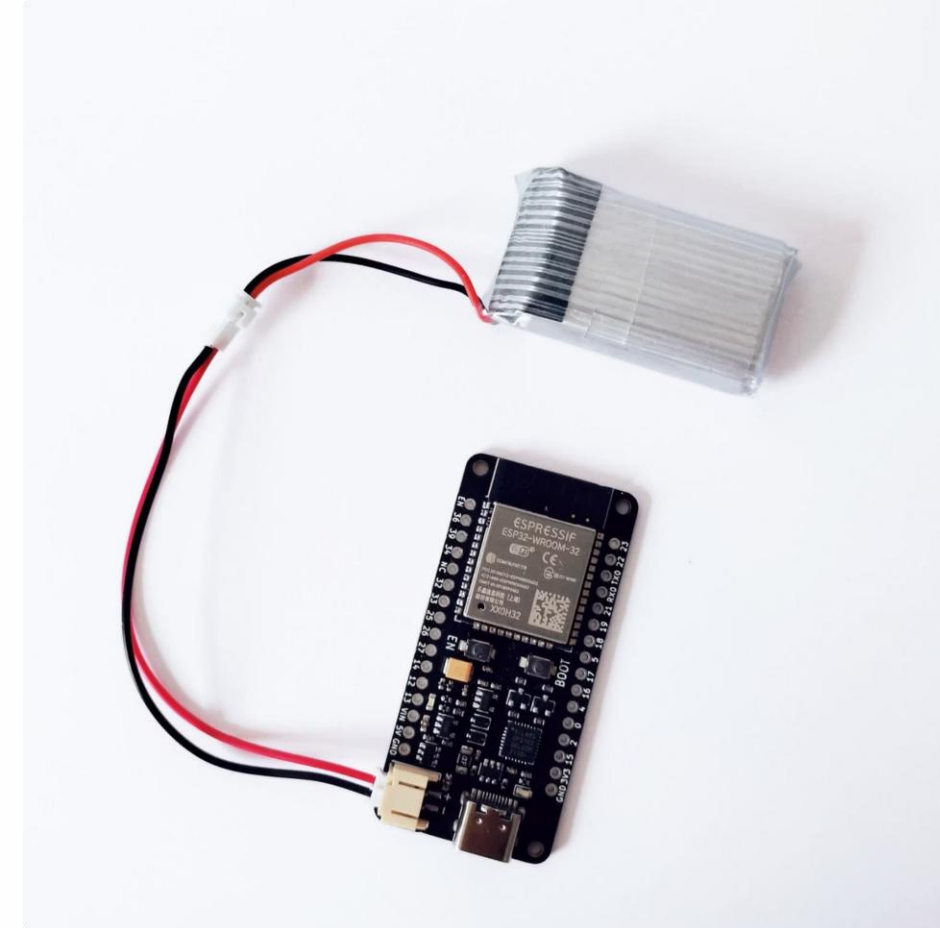


Low-Cost-Prototyping

Sehr günstiger Einstieg für verteilte IoT-Knoten. Großes Ökosystem mit MicroPython, Arduino-Framework und ESP-IDF.



Der ESP32 ist kein kleiner Linux-Rechner, sondern ein vernetzter Embedded-Controller mit sehr gutem Preis-Leistungs-Verhältnis. Kosten ab 5€



ESP32: Technische Grenzen

Ressourcengrenzen

- **Begrenzte RAM- und Flash-Ressourcen**
(typisch: einige hundert KB SRAM)
- **Keine typische Desktop-/Server-Laufzeitumgebung**
- **Eingeschränkte Parallelität** komplexer Software-Stacks
- Begrenzte Eignung für aufwendige Bild- oder KI-Verarbeitung

Architektonische Grenzen im IoT

- Komplexe Container-Workloads ungeeignet
- Hohe Datenpersistenz lokal schwierig realisierbar
- Multimedia und anspruchsvolle GUI unpraktisch
- Software-Wartbarkeit bei wachsender Komplexität begrenzt
- Echtzeitnähe möglich, aber ohne Linux-Userspace-Komfort

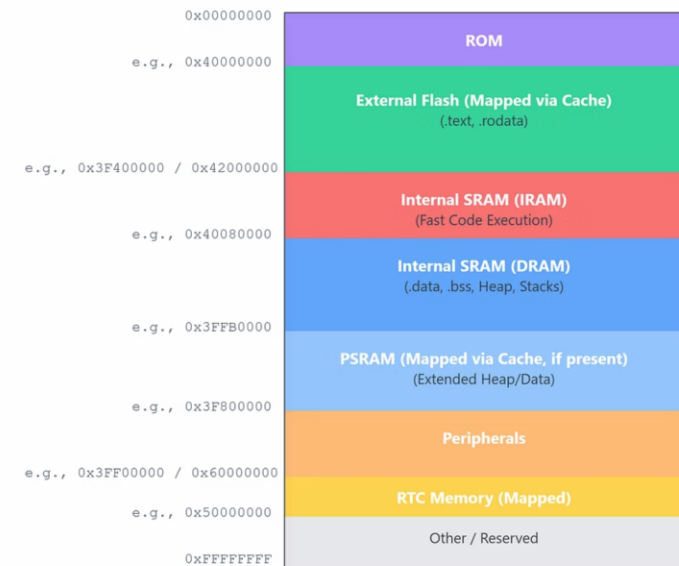


Zwischen „technisch möglich“ und „architektonisch sinnvoll“ ist sauber zu unterscheiden.

RAM – Random Access Memory · SRAM – Static Random Access Memory · Flash – Flash-Speicher (nichtflüchtig) · KB – Kilobyte · RTOS – Real-Time Operating System · GUI – Graphical User Interface · IoT – Internet of Things · MCU – Microcontroller Unit

Simplified ESP32 Memory Map Concept (Not to Scale)

(Address ranges are illustrative and vary by ESP32 variant)



Consult specific ESP32 variant TRM for actual addresses.

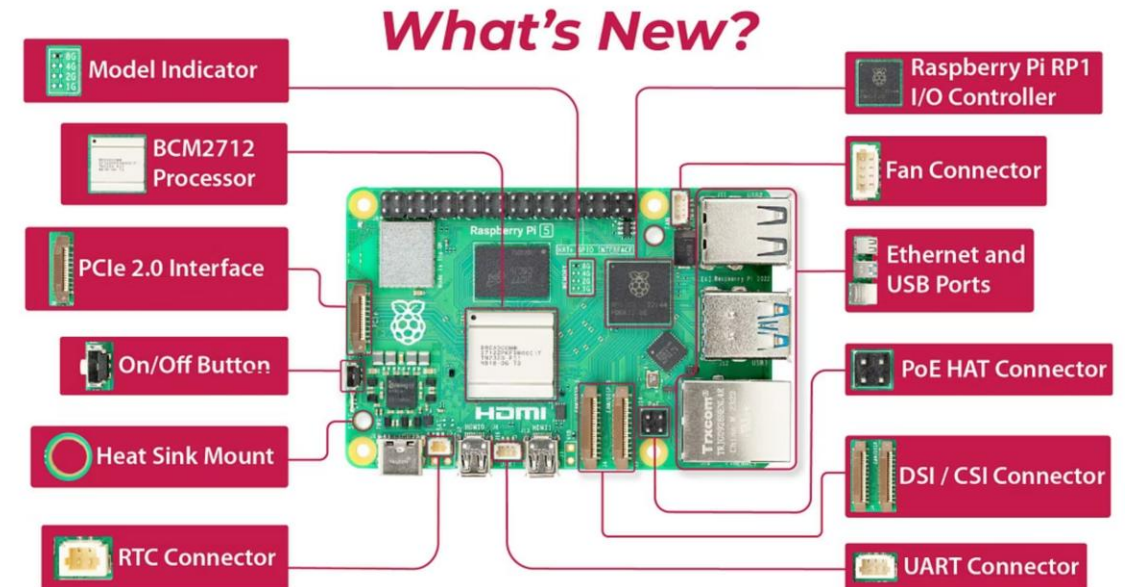
Architektur des Raspberry Pi 5

Recheneinheit und Speicher

- **Quad-Core Arm Cortex-A76** mit 2,4 GHz
- **LPDDR4X-RAM** je nach Modell (4 GB / 8 GB)
- **VideoCore VII GPU**
- Linux-fähige Plattform – Raspberry Pi OS, Ubuntu u. a.

Schnittstellen

- **40-Pin GPIO** – kompatibel zu HAT-Erweiterungen
- **USB 2.0 und USB 3.0**
- **Gigabit Ethernet**
- WLAN im 2,4- und 5-GHz-Band, Bluetooth
- Micro-HDMI, Kamera (CSI), Display (DSI), M.2 (PCIe)



Der Raspberry Pi 5 markiert mit PCIe-Anbindung und neuem South-Bridge-Chip (RP1) einen Generationssprung in der I/O-Architektur → Schneller Schnittstellen

Raspberry Pi: Was kann das Gerät gut?



Vollwertiger Software-Stack

GNU/Linux, Treiber, Bibliotheken, Netzwerkdienste und containernahe Workflows (Docker, Podman).

Höhere **Rechenleistung**

Edge Analytics, Gateway-Funktionen, Protokollübersetzung und lokale Datenvorverarbeitung.

Breites Ökosystem

Große Community, viele Bibliotheken, Kamera-HATs, Sensor-Erweiterungen und umfassende Dokumentation.

Schnittstellenvielfalt

Netzwerk, USB, Massenspeicher, Display und GPIO auf einer Plattform – skalierbar durch HATs.

 Raspberry Pi eignet sich besonders dort, wo IoT-Knoten mehr als reine Steuergeräte sein sollen.



RPi – Raspberry Pi · SBC – Single-Board Computer · IoT – Internet of Things · HAT – Hardware Attached on Top · GPIO – General Purpose Input/Output · USB – Universal Serial Bus · MQTT – Message Queuing Telemetry Transport · HTTP – Hypertext Transfer Protocol · Docker – Container-Laufzeitumgebung · Podman – Container-Engine

Raspberry Pi: Technische Grenzen

Trotz deutlich höherer Rechenleistung bringt der Raspberry Pi spezifische Einschränkungen mit, die für IoT-Systeme architektonisch relevant sind.

Energiebedarf und Laufzeit

Höherer Energiebedarf als Mikrocontrollerplattformen. Geringere Eignung für langlaufende Batterieknoten. Bootzeit und Systemkomplexität deutlich höher als bei MCU-basierten Boards.

Determinismus und Robustheit

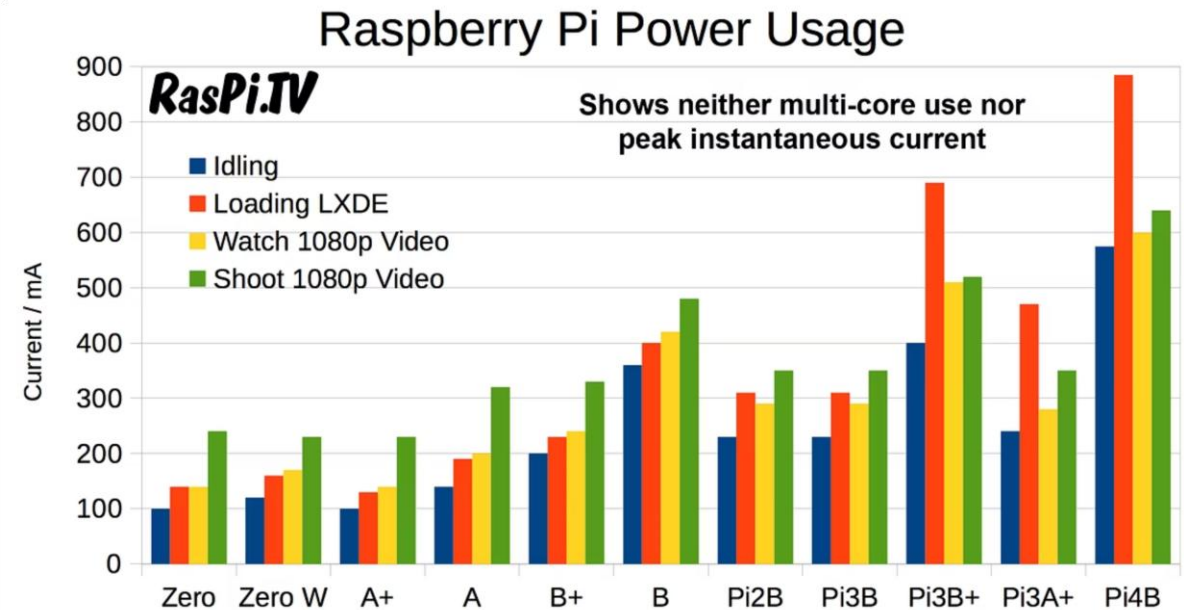
Weniger deterministisches Zeitverhalten unter Linux (kein RTOS-Hardgarantie). SD-Karten-basierte Systeme erzeugen Robustheitsprobleme im Feldeinsatz. Höherer Administrations- und Wartungsaufwand.

Überdimensionierung

Für sehr einfache Mess- und Schaltaufgaben oft überdimensioniert. Höhere Kosten pro Knoten und mehr Wartungsaufwand rechtfertigen den Einsatz nur bei entsprechend komplexen Anforderungen.



Mehr Leistung löst nicht automatisch IoT-Probleme; oft erhöht sie nur Kosten, Energiebedarf und Wartungsaufwand.



Vergleichstabelle: ESP32 und Raspberry Pi im IoT

Merkmal	ESP32	Raspberry Pi 5
Rechenmodell	MCU / SoC	SBC (Arm Cortex-A76)
Betriebssystem	RTOS / Bare Metal typisch	Linux typisch
Energiebedarf	Niedrig (mW-Bereich)	Deutlich höher (W-Bereich)
Echtzeit-/Steuerungsnahe	Hoch	Geringer deterministisch
Funk / Netzwerk	WLAN + Bluetooth integriert	WLAN, BT, Gigabit Ethernet
Speicherressourcen	Stark begrenzt (KB-Bereich)	Deutlich höher (GB-Bereich)
Software-Komplexität	Moderat bis niedrig	Hoch
Typische Rolle	Sensorknoten, Embedded Control	Gateway, Edge Node, Analyse
Grenzen	Begrenzte Rechen- und Speicherressourcen	Mehr Energie, Wartung, weniger echtzeitnah

MCU – Microcontroller Unit · SoC – System on a Chip · SBC – Single-Board Computer · RTOS – Real-Time Operating System · WLAN – Wireless Local Area Network · BT – Bluetooth · GB – Gigabyte · KB – Kilobyte · IoT – Internet of Things

Vergleichstabelle: ESP32 und anderen Geräten

ESP32 vs Competitors



ESP32

Arduino

STM32

Raspberry
Pi Pico

Nordic
nRF52

Wi-Fi	✓	✗	✗	✗	✗
Bluetooth	✓	✗	✓	✓	✓
Dual-Core	✓	✓	✓	✓	✓
Low Power	✓	✓	Easy	Eay	Medium
Ease of Use	Medium	Easy	Medium (-\$2-10+)	Very low (-\$4-10)	Medium (\$5-\$15)
Price	Low (-\$5-10)	Low (-\$5-20)	Low	Low	Med
TinyML / AI	(S3)	Low	Low	High	Medc

Arduino-Plattform im IoT

Mikrocontroller-Klasse · 8-bit AVR / 32-bit ARM (Due, Zero)

Architektur

- ATmega328P (Uno): 8-bit, 16 MHz, 32 KB Flash, 2 KB SRAM → Sehr klein
- ARM-Varianten (Due, Zero): 32-bit, 84 MHz, 512 KB Flash
- Kein integriertes WLAN/BT → externe Module (ESP8266, nRF24L01)

Stärken

- Extrem niedrige Einstiegshürde, riesige Community
- Tausende Shields & Bibliotheken
- Deterministisches Timing, kein OS-Overhead
- Ideal: einfache Sensor-/Aktoraufgaben, Prototyping

Grenzen

- Kein nativer IP-Stack / Netzwerk-Stack
- Sehr begrenzter Speicher (kein komplexes JSON-Parsing)
- Kein Multitasking (nur kooperatives Scheduling)
- Nicht geeignet für Edge-Analytics oder Gateway-Funktionen

IoT-Relevanz

- Einsatz: Bildung, Maker-Projekte, einfache Feldknoten
- Protokolle: UART-Bridge zu ESP, SPI/I²C zu Sensoren
- Typische Anwendung: Temperaturlogger, Aktorsteuerung



MCU – Microcontroller Unit · AVR – Alf and Vegard's RISC processor · ARM – Advanced RISC Machine · SRAM – Static Random Access Memory · SPI – Serial Peripheral Interface · I²C – Inter-Integrated Circuit · UART – Universal Asynchronous Receiver-Transmitter · WLAN – Wireless Local Area Network · BT – Bluetooth

STM32-Plattform im IoT

Mikrocontroller-Klasse · 32-bit ARM Cortex-M (STMicroelectronics)

Architektur

- Cortex-M0 bis M7: 48–480 MHz, bis 2 MB Flash, bis 1 MB SRAM
- FPU, DMA, **Hardware-Krypto auf höheren Varianten**
- Integrierte Peripherie: CAN, USB, Ethernet, SDIO, ADC (12-bit)

Stärken

- **Echtzeitfähigkeit: FreeRTOS, Zephyr, ThreadX** nativ unterstützt
- Präzise Peripheriesteuerung via HAL / LL-Treiber
- **Industriell erprobt**, langer Produktlebenszyklus
- Niedrigster Energieverbrauch im Vergleich (Stop-/Standby-Modi)

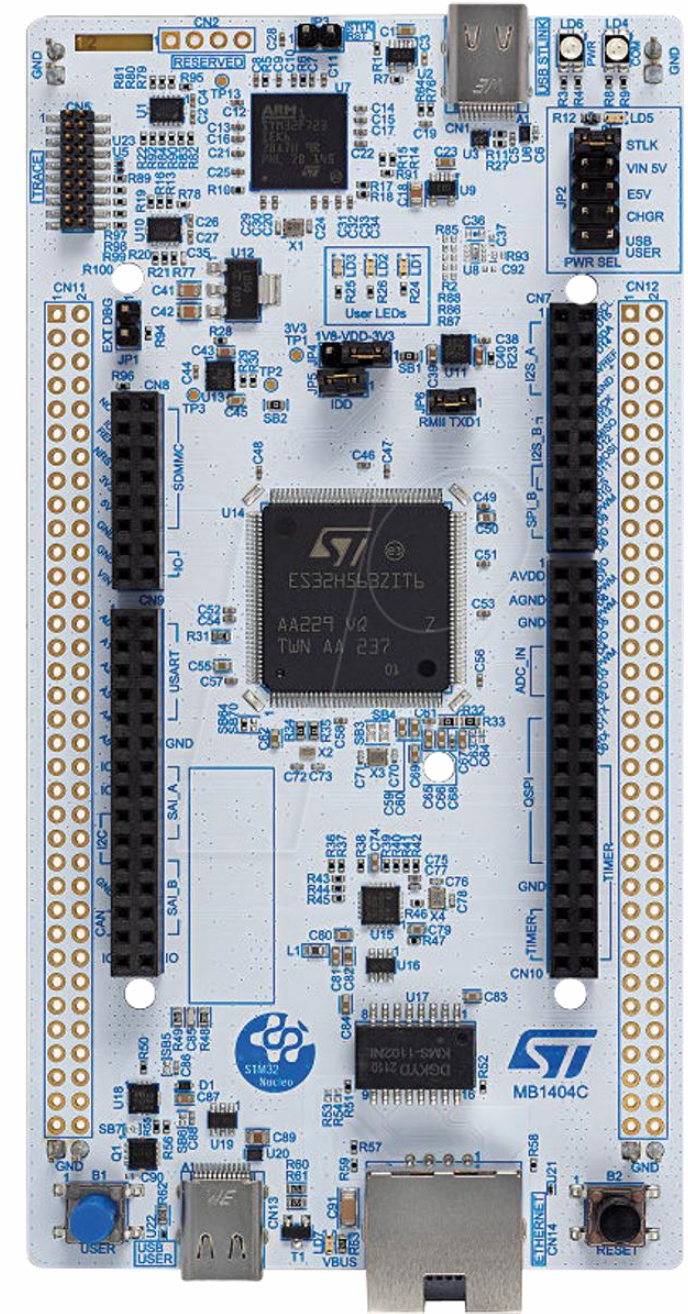
Grenzen

- Steile Lernkurve
- Kein integriertes WLAN/BT (Ausnahme: STM32WB, STM32WL)
- Debugging-Setup aufwändiger als Arduino/ESP32
- Kleinere Maker-Community als Arduino oder ESP32

IoT-Relevanz

- Einsatz: Industrieautomation, Medizintechnik, Automotive
- Protokolle: CAN-Bus, Modbus, MQTT über externe Netzwerkmodule
- Typische Anwendung: Motorsteuerung, Sensorknoten mit Echtzeit-Anforderungen

MCU – Microcontroller Unit · ARM – Advanced RISC Machine · FPU – Floating Point Unit · DMA – Direct Memory Access · CAN – Controller Area Network · HAL – Hardware Abstraction Layer · LL – Low-Layer · RTOS – Real-Time Operating System · FreeRTOS – Free Real-Time Operating System · SDIO – Secure Digital Input Output · ADC – Analog-to-Digital Converter · WLAN – Wireless Local Area Network · BT – Bluetooth · MQTT – Message Queuing Telemetry Transport



BeagleBone-Plattform im IoT

Single-Board-Computer-Klasse · ARM Cortex-A + PRU-Subsystem

Architektur

- AM335x / AM5729: Cortex-A8/A15, 1–1,5 GHz
- Einzigartiges PRU-Subsystem: 2× 200 MHz Echtzeit-Coprozessoren
- Onboard eMMC (4 GB), kein SD-Karten-Pflichtbetrieb
- **65+ GPIO**, 7× ADC (12-bit), 4× UART, 2× SPI, 2× I²C

Stärken

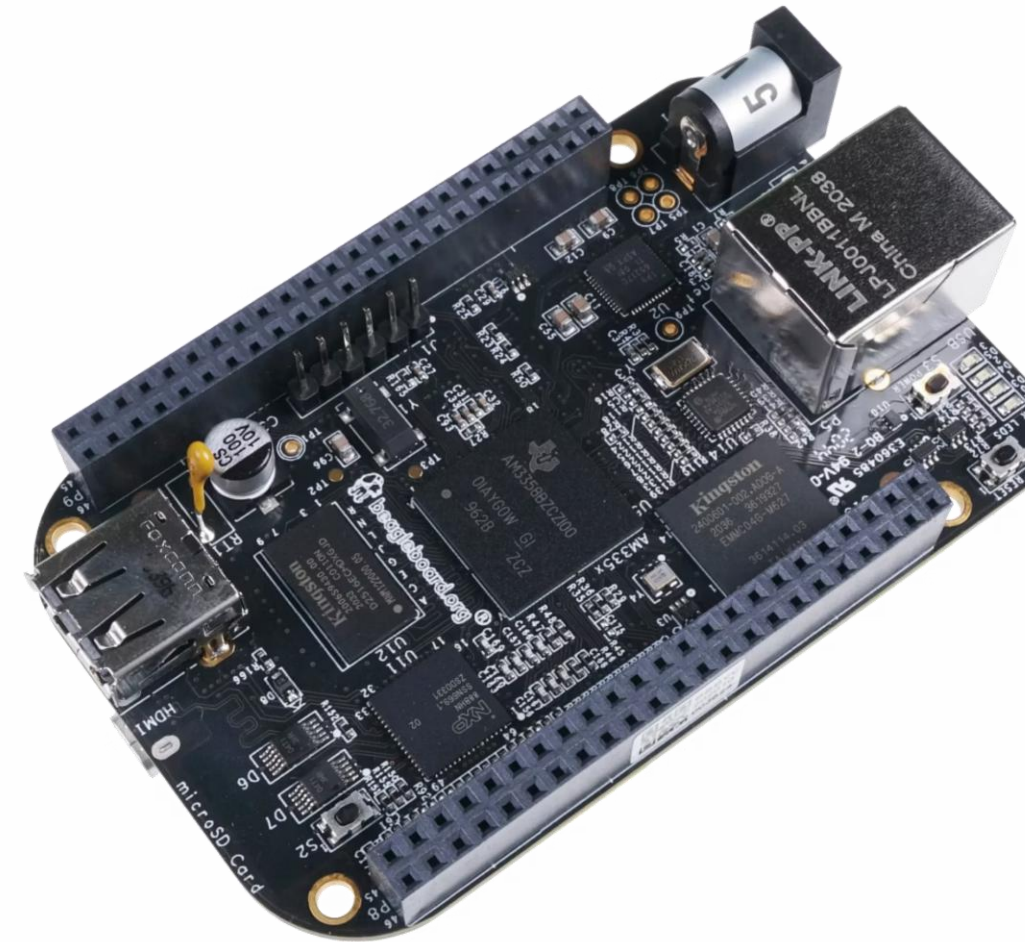
- PRU ermöglicht deterministisches Echtzeit-I/O unter Linux
- Kein SD-Karten-Robustheitsproblem (eMMC)
- Industriell ausgerichtet: langer Support-Zyklus, robuste Peripherie
- Vollwertiges Linux (Debian) + Echtzeit-Fähigkeit kombiniert

Grenzen

- Kleinere Community als Raspberry Pi oder Arduino
- PRU-Programmierung komplex (C, Assembly)
- Höherer Energiebedarf als reine MCU-Boards
- Weniger HAT-/Cape-Ökosystem als Raspberry Pi

IoT-Relevanz

- Einsatz: Industrielle Edge-Knoten, Robotik, Motorsteuerung mit Linux
- Protokolle: EtherCAT, CAN, MQTT, Modbus über PRU oder Linux-Stack
- Typische Anwendung: CNC-Steuerung, industrielle Gateway-Knoten



SBC – Single-Board Computer · PRU – Programmable Real-time Unit · ARM – Advanced RISC Machine · GPIO – General Purpose Input/Output · ADC – Analog-to-Digital Converter · UART – Universal Asynchronous Receiver-Transmitter · SPI – Serial Peripheral Interface · I²C – Inter-Integrated Circuit · eMMC – embedded MultiMediaCard · MCU – Microcontroller Unit · CAN – Controller Area Network · MQTT – Message Queuing Telemetry Transport · EtherCAT – Ethernet for Control Automation Technology

Kommunikationsschnittstellen und Peripherie

Hardware-nahe Busse (Platinenebene)

- **GPIO** – digitale Ein-/Ausgabe, Interrupts
- **I2C** – synchroner 2-Draht-Bus, Sensor-Anbindung
- **SPI** – schneller synchroner Bus, Display/Flash
- **UART** – serielle Kommunikation, Debugging
- **PWM** – analoge Signalerzeugung, Motorsteuerung
- **ADC / DAC** – Analog-Digital- und Digital-Analog-Wandlung

Diese Schnittstellen entscheiden über die Anbindung realer Sensorik und Aktorik.

Netzwerkanbindung (Netzebene)

- **WLAN** – IEEE 802.11, häufigste IoT-Konnektivität
- **Bluetooth / BLE** – Kurzstreckenkommunikation
- **Ethernet** – kabelgebunden, zuverlässig für Gateways
- **LoRa / LoRaWAN** – Langstrecke, sehr geringer Energiebedarf
- **Mobilfunk (LTE-M, NB-IoT)** – über Zusatzmodule

Für IoT-Architekturen ist relevant, ob Kommunikation lokal, gateway-basiert oder cloud-direkt erfolgt.



Protokolle auf Platinenebene (I2C, SPI) und Netzebene (MQTT, HTTP) klar unterscheiden.

GPIO – General Purpose Input/Output · I2C – Inter-Integrated Circuit · SPI – Serial Peripheral Interface · UART – Universal Asynchronous Receiver-Transmitter · PWM – Pulse Width Modulation · ADC – Analog-to-Digital Converter · DAC – Digital-to-Analog Converter · WLAN – Wireless Local Area Network · BLE – Bluetooth Low Energy · LoRa – Long Range (Funktechnologie) · LoRaWAN – Long Range Wide Area Network · LTE-M – LTE for Machines · NB-IoT – Narrowband IoT · MQTT – Message Queuing Telemetry Transport · HTTP – Hypertext Transfer Protocol

Software-Stack im Vergleich

Bare Metal

Maximale Hardwarekontrolle, minimale Abstraktion. Direkter Registerzugriff, deterministische Ausführung, kein Overhead durch Betriebssystem.

RTOS auf MCUs

Planbares Tasking (FreeRTOS, Zephyr), geringe Latenz, knappe Ressourcen. Ermöglicht nebenläufige Tasks bei definiertem Zeitverhalten.

Embedded Middleware

Treiber, Netzwerk-Stacks, MQTT/HTTP-Clients, OTA-Mechanismen. Abstraktionsschicht zwischen Hardware und Anwendungslogik.

Linux auf SBCs

Prozesse, Pakete, Dateisysteme, Container, Edge Services. Volle Softwareflexibilität bei erhöhter Systemkomplexität und Energiebedarf.

 Die Wahl der Hardware ist immer auch eine Wahl des Softwaremodells – und umgekehrt.

RTOS – Real-Time Operating System · MCU – Microcontroller Unit · SBC – Single-Board Computer · FreeRTOS – Free Real-Time Operating System · Zephyr – Open-Source RTOS · MQTT – Message Queuing Telemetry Transport · HTTP – Hypertext Transfer Protocol · OTA – Over-The-Air (Update) · IoT – Internet of Things · OS – Operating System

What is Full Stack in Embedded Software?

Here's how Full Stack Embedded looks – explained with a car analogy!

1. Drivers (C, Assembly)

Like wiring your car's engine and sensors directly to the control system.

 Handles low-level hardware (UART, SPI, GPIO, etc.)



2. RTOS/Bare-metal Logic

Like the gearbox – managing how tasks are scheduled and prioritized.

 Tasks, Interrupts, Timers, Semaphores, etc.



3. Middleware / Protocol Stack

Like your car's transmission system – it enables communication.

 Protocols like CAN, Modbus, Bluetooth, LoRa, Wi-Fi, MQTT, etc.



4. Application Logic

The dashboard – your speedometer, fuel indicators, warning lights.

 Business logic that processes data, handles UI (LCD/LED), etc.



5. Cloud / IoT Gateway

The mobile app that shows car speed remotely

 Integration with AWS IoT, Azure, or a local gateway



Sicherheit und Wartbarkeit

Sicherheitsrelevante Aspekte

- **Secure Boot** – Verifikation der Firmware beim Start
- **Flash-/Datenträgerverschlüsselung** – Schutz gespeicherter Daten
- **Schlüsselmanagement** – sichere Ablage von Credentials
- **Sichere Update-Strategien (OTA)** – signierte Firmware-Images
- **Härtung von Netzwerkschnittstellen** – Dienste minimieren

Wartbarkeit im IoT-Betrieb

- **OTA-Updates** – Firmware über Netz aktualisieren
- **Fernkonfiguration** – Parameter ohne physischen Zugriff ändern
- **Logging und Monitoring** – Fehlerzustände frühzeitig erkennen
- **Fehlertoleranz** – Watchdog, Reset-Strategien, Persistenz

Ein Raspberry Pi bietet mehr Komfort für komplexe Wartungs-Stacks; ein ESP32 bietet durch seinen geringeren Softwareumfang eine geringere Angriffsfläche.

❌ Sicherheit ist kein Add-on, sondern ein Architekturmerkmal – von Beginn an einzuplanen.

OTA – Over-The-Air (Update) · IoT – Internet of Things · MCU – Microcontroller Unit · SBC – Single-Board Computer · RPi – Raspberry Pi · ESP32 – Espressif Systems 32-bit SoC · JTAG – Joint Test Action Group · TLS – Transport Layer Security · PKI – Public Key Infrastructure

How Does an OTA Update Work For IoT Devices?



1. A device issues the updates



2. The update is sent to the IoT device through the cloud



3. The update is downloaded to the IoT device

Typische IoT-Einsatzszenarien

ESP32: Batteriebetriebene Sensorik

Raumklima- und Umweltmessung, BLE/WLAN-nahe Edge-Knoten, einfache Steuer- und Schaltlogik. Geringe Kosten, langer Batteriebetrieb.



Raspberry Pi: Edge Gateway

Smart-Home-Zentrale, industrielles Protokoll-Gateway, Kamera- und Audioverarbeitung, lokales Daten-Caching und Edge Analytics.



Der Plattformfit ergibt sich aus Lastprofil, Energieversorgung, Konnektivität und Wartungsmodell.



Architektur-Trade-offs bei der Plattformwahl

Zentrale Abwägungsdimensionen

Energiebedarf vs.
Rechenleistung

Determinismus vs.
Softwarekomfort

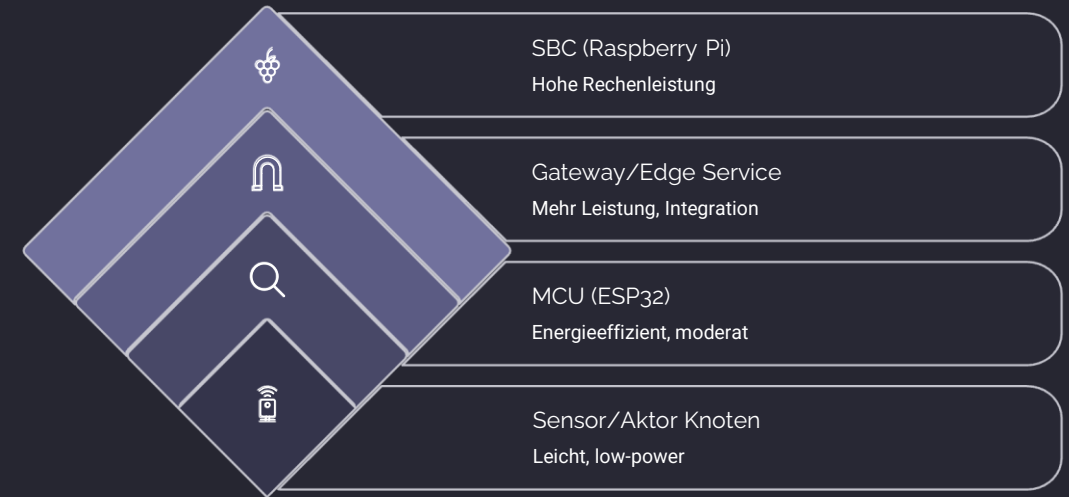
Integrationsgrad vs.
Erweiterbarkeit

Kosten pro Knoten vs.
Funktionsumfang

Robustheit im Feld vs. Entwicklungsbequemlichkeit

Entscheidungslogik

In IoT-Systemen ist die beste Plattform selten die leistungsstärkste, sondern die architektonisch angemessenste.



Kombinierte Architekturen: ESP32 + Raspberry Pi

Hybride IoT-Architektur

Viele reale IoT-Systeme nutzen nicht eine Plattform, sondern eine abgestufte Hardwarearchitektur mit klarer Rollentrennung:

- **ESP32** als feldnaher Sensorknoten mit geringem Energiebedarf
- **Raspberry Pi** als lokales Gateway oder Edge-Server

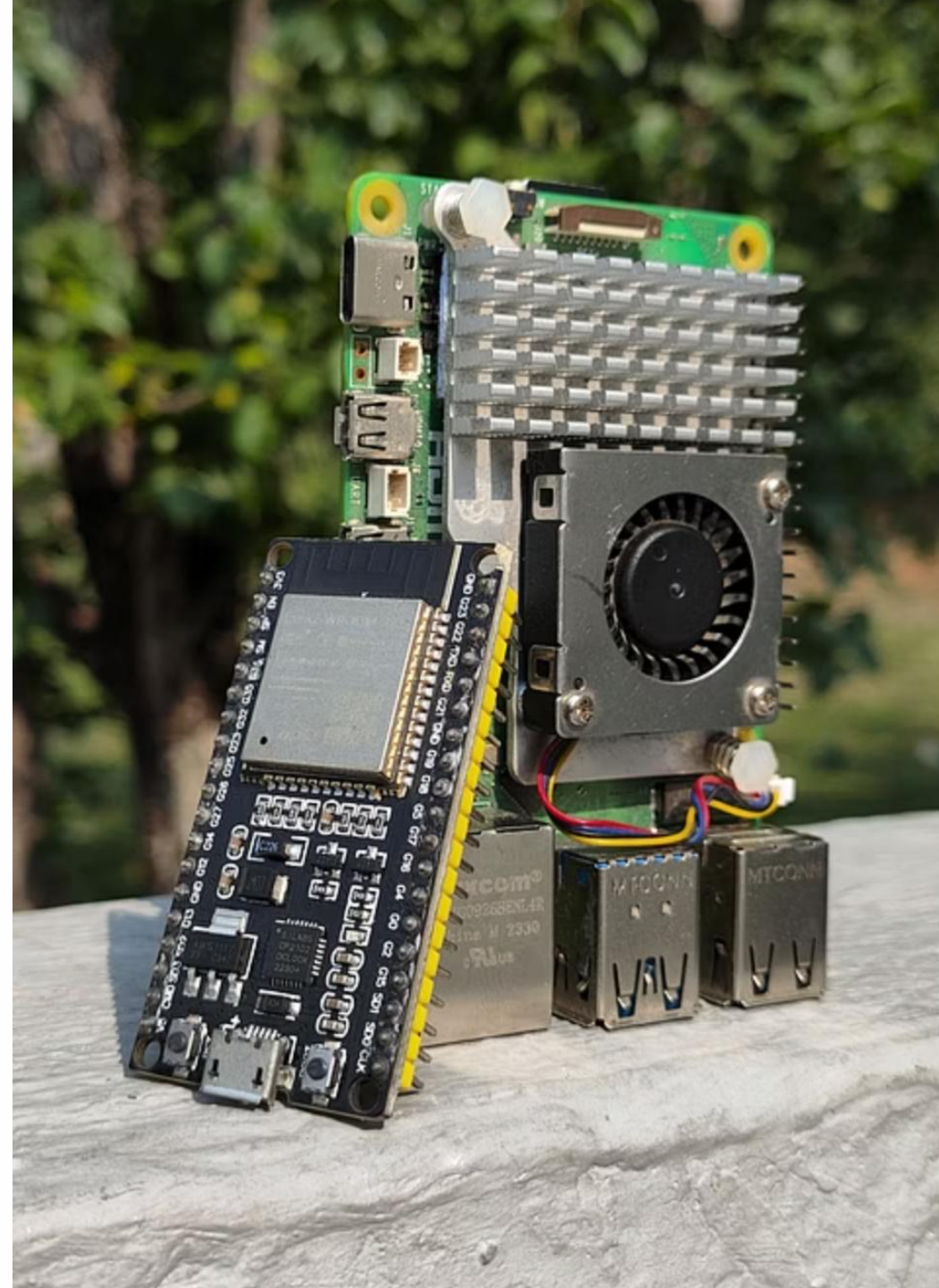
Vorteile: Lasttrennung, bessere Energieeffizienz und saubere funktionale Rollentrennung.

Typische Funktionsverteilung

- **ESP32:** Messen, Vorfiltern, Schalten, Funkzugang (BLE/WLAN)
- **Raspberry Pi:** Aggregation, Persistenz, Dashboard, Protokollkonversion, Cloud-Anbindung

Diese Kombination erlaubt es, Energiebudget und Rechenressourcen optimal entlang der Systemschichten zu verteilen.

ESP32 – Espressif Systems 32-bit SoC · RPi – Raspberry Pi · IoT – Internet of Things · BLE – Bluetooth Low Energy · WLAN – Wireless Local Area Network · SBC – Single-Board Computer · MCU – Microcontroller Unit · Edge Server – lokaler Verarbeitungsknoten am Netzwerkrand



Kernaussagen

ESP32 und ähnliche MCU-Boards

Stark bei energieeffizienter, feldnaher, echtzeitnaher IoT-Steuerung. Optimale Wahl für Sensorknoten, Aktorik und batteriebetriebene verteilte Systeme mit begrenztem Softwarestack.

Raspberry Pi und ähnliche SBCs

Stark bei komplexeren Edge- und Gateway-Aufgaben mit Linux-basiertem Software-Stack. Geeignet für Protokollkonversion, lokale Analyse und containernahe Workflows.

Die zentrale technische Frage

Nicht „welches Board ist besser?“, sondern: Welche Ressourcen, Latenzen, Sicherheitsanforderungen, Wartungsmodelle und Kosten sind im Zielsystem relevant? Gute IoT-Architekturen entstehen durch saubere Plattformwahl und klare funktionale Rollen.

❓ Diskussion: Wo liegen die Grenzen einer reinen MCU-Architektur, und ab wann ist ein SBC als Edge-Schicht architektonisch sinnvoll?

ESP32 – Espressif Systems 32-bit SoC · MCU – Microcontroller Unit · SBC – Single-Board Computer · RPi – Raspberry Pi · IoT – Internet of Things · RTOS – Real-Time Operating System · BLE – Bluetooth Low Energy · WLAN – Wireless Local Area Network · Edge – Netzwerkrand · Gateway – Protokoll-/Netzwerkbrücke

Key Components of an IoT Technology Stack



Sensors



Connectivity



Cloud Platform



Analytics

Internet der Dinge – Internet of Things: IoT

Wir wollen unsere Geräte (Sensor, Mikrocontroller, PCs...) miteinander verbinden.

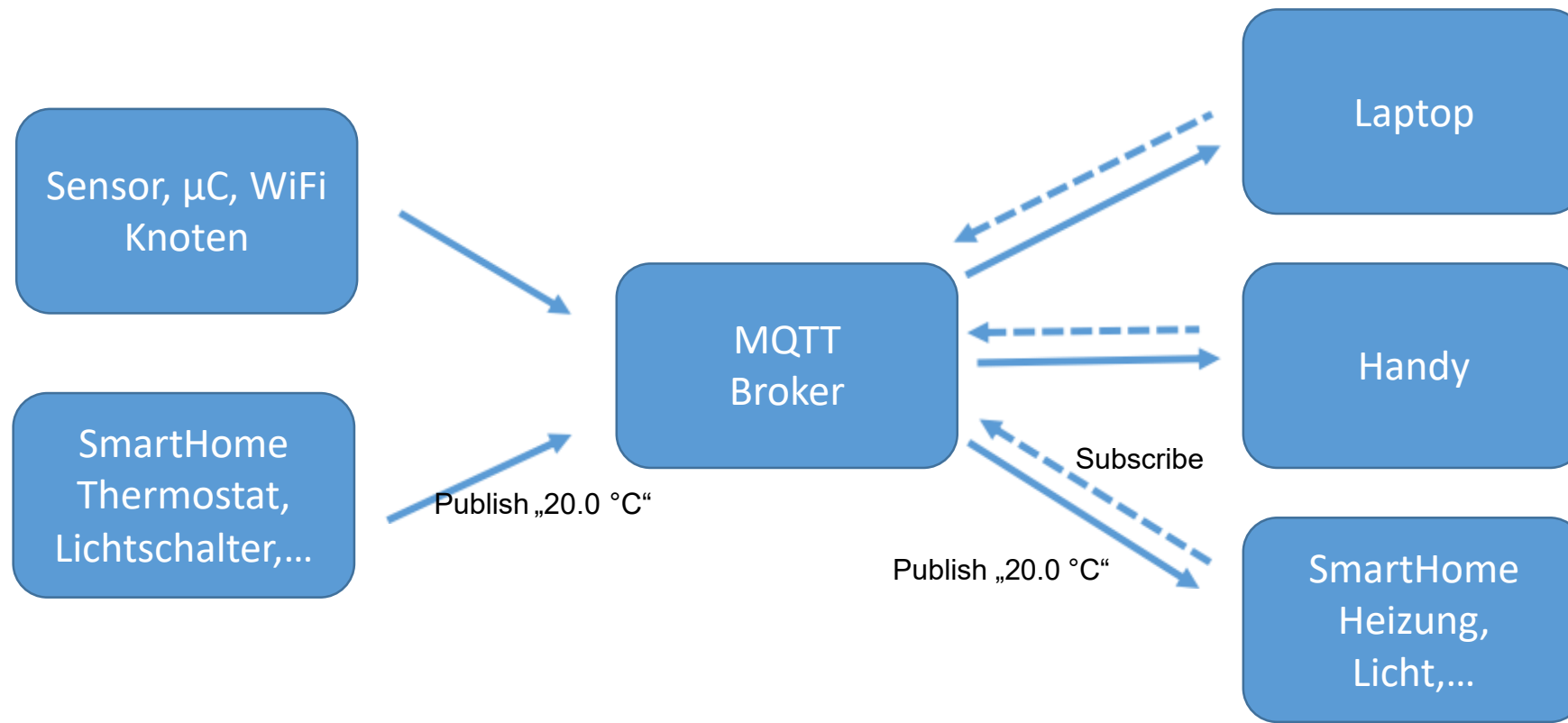
Wie machen wir das?

Bausteine für das IoT:

- MQTT –Message Queue TelemetryTransport
- Node-Red–<https://nodered.org/about/>
- JSON –JavaScript Object Notation
- openHAB–open Home Automation Bus
- u.v.m...

MQTT – Message Queue Telemetry Transport

MQTT: Publish/Subscribe



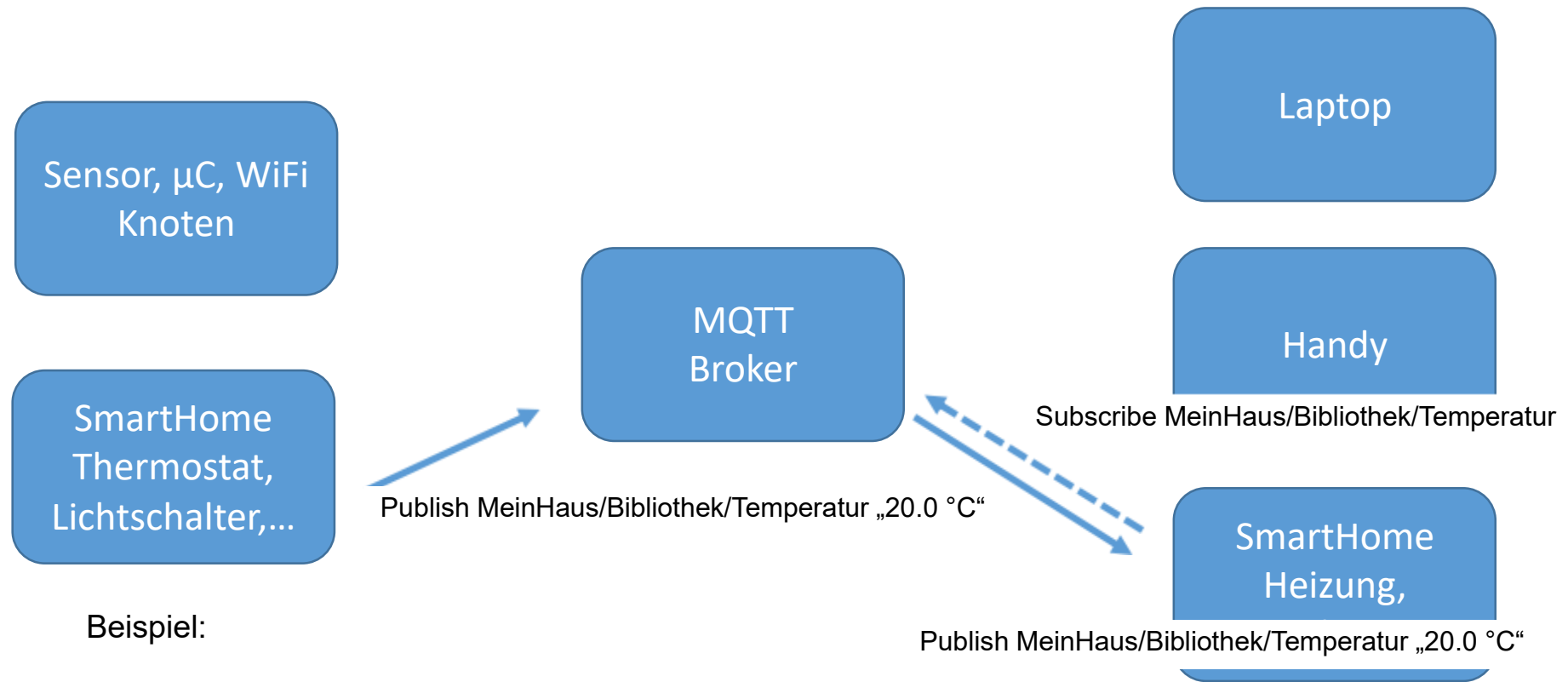
Im Gegensatz dazu HTTP: Request/Response

Bild nach: Walter Trokan, Das MQTT-Praxisbuch

MQTT – Message Queue Telemetry Transport

Adressierung über Topic („Betreff“)

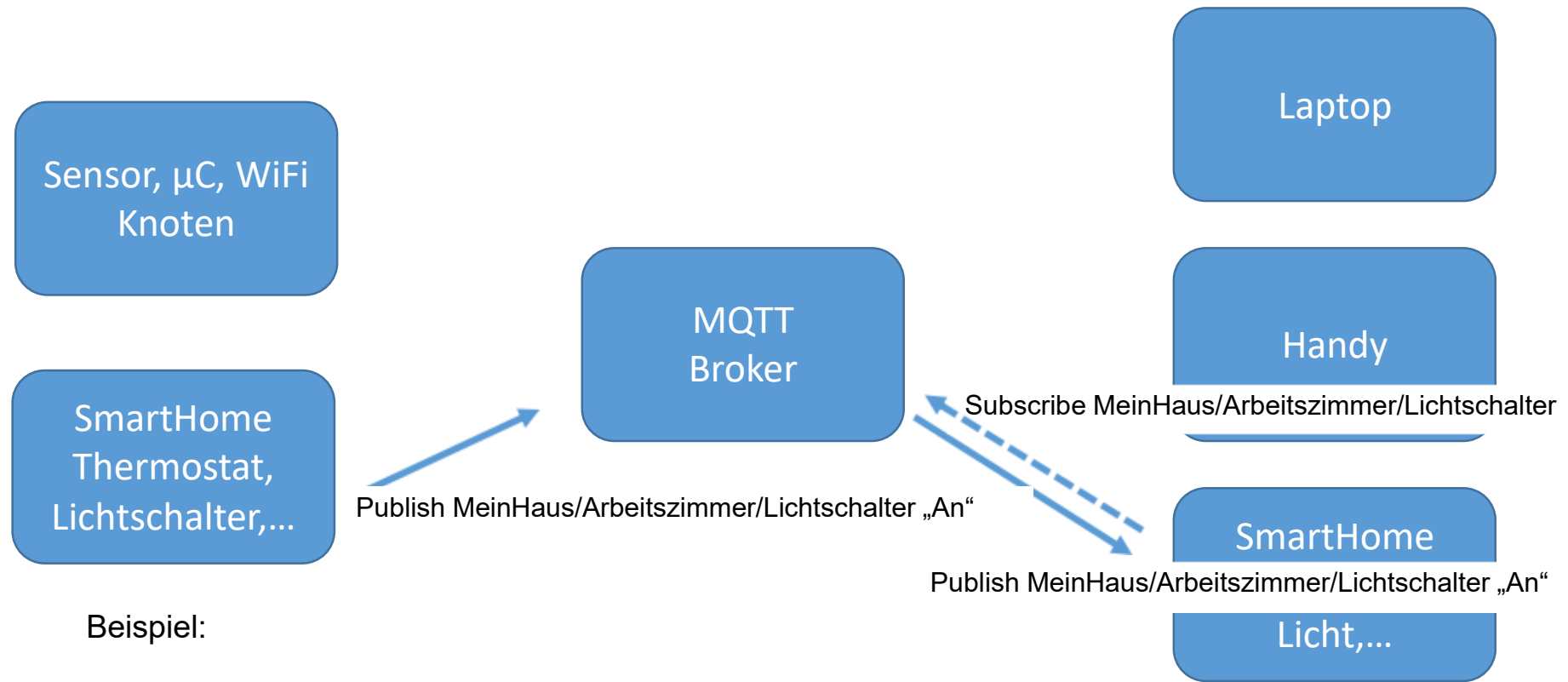
Nutzdaten als „Payload“



MQTT – Message Queue Telemetry Transport

Adressierung über Topic („Betreff“)

Nutzdaten als „Payload“

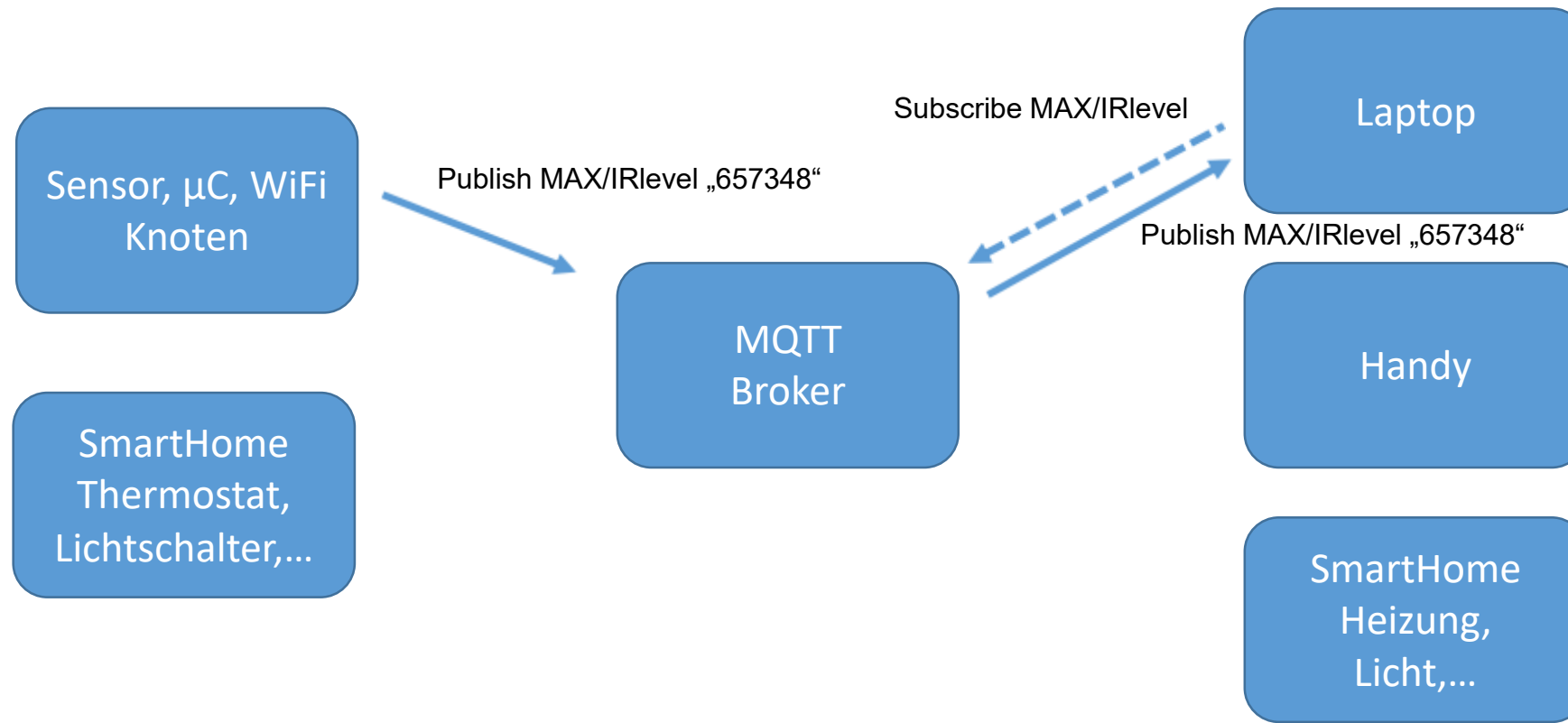


MQTT – Message Queue Telemetry Transport

„Wildcards“:

Single Level: MeinHaus/+/Temperatur

Multi Level: MeinHaus/#



MQTT – Qualitätssicherung

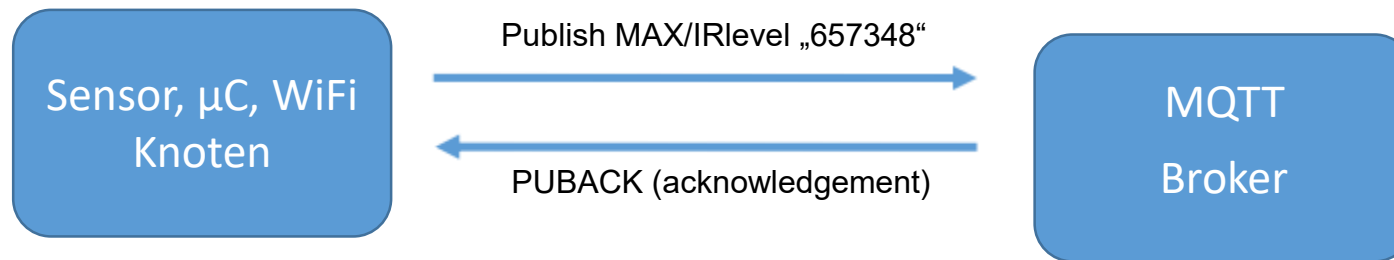
QoS0: Maximal einmal



Geringste Netzbelastung
Weder Empfangsbestätigung
noch Zwischenspeicherung beim Sender

MQTT – Qualitätssicherung

QoS1: Mindestens einmal

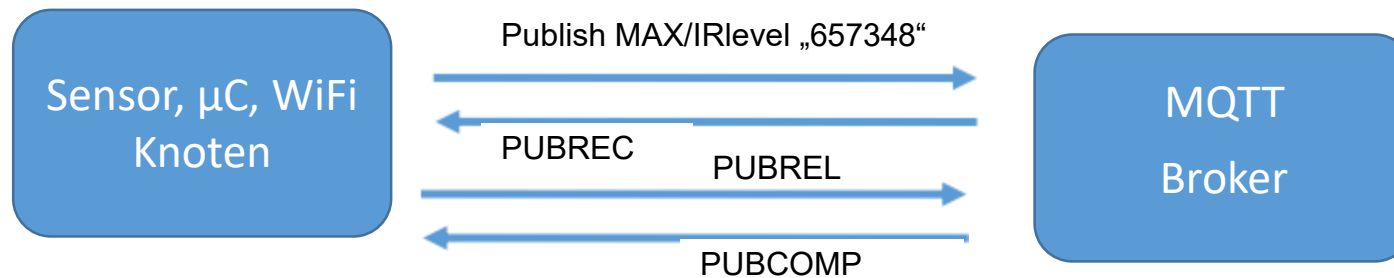


Moderate Netzbelastung
Empfangsbestätigung
Bis dahin Zwischenspeicherung beim Sender

Sender sendet bei ausbleibender Empfangsbestätigung mehrmals
Fehlermöglichkeit: z.B. Lichtschalter wird mehrmals betätigt

MQTT – Qualitätssicherung

QoS2: Garantiert nur einmal



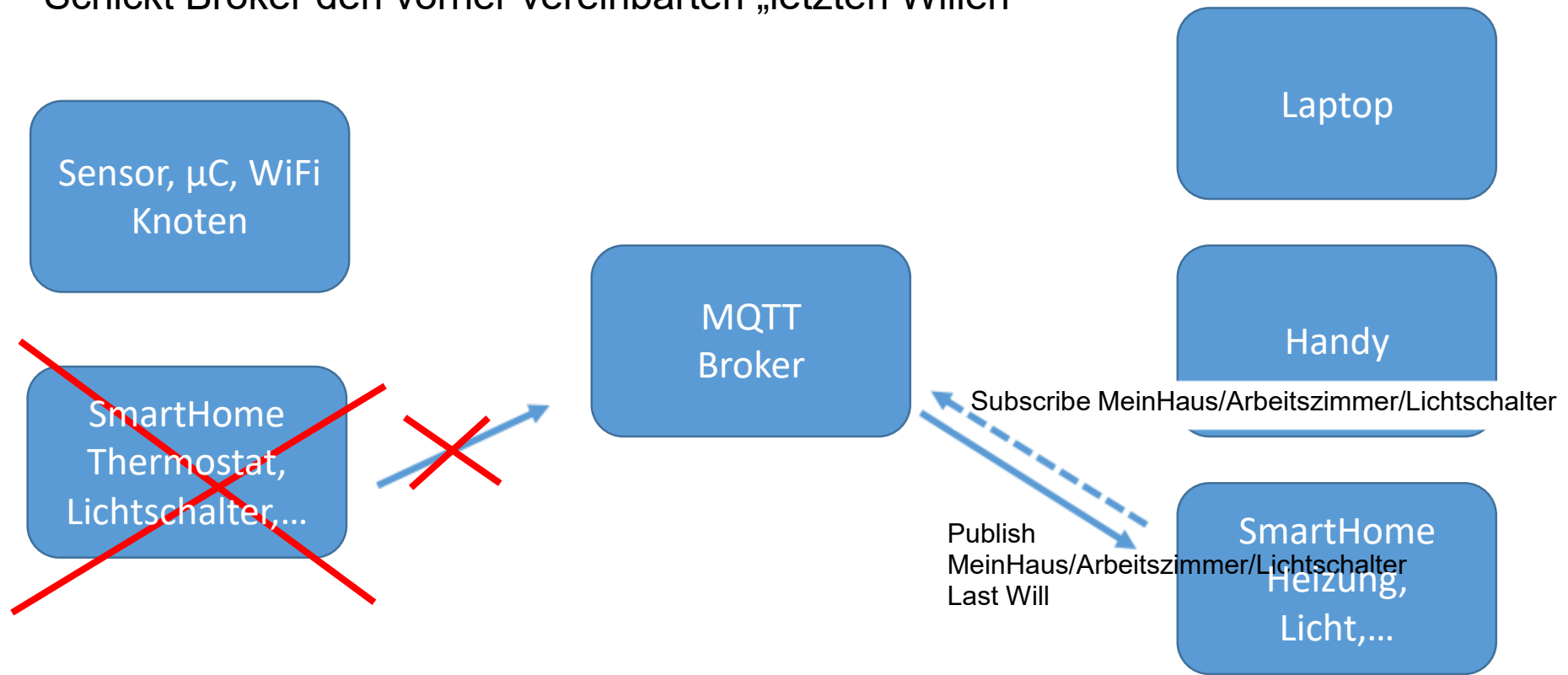
Moderate Netzbelastung
Empfangsbestätigung
Bis dahin Zwischenspeicherung beim Sender

Sender sendet bei ausbleibender Empfangsbestätigung mehrmals
Fehlermöglichkeit: z.B. Lichtschalter wird mehrmals betätigt

MQTT – LetzterWille/ Testament

Wenn Sensor „abnibbelt“

Schickt Broker den vorher vereinbarten „letzten Willen“



MQTT

Ausprobieren! Auf wokwi.com/esp32

Online ESP32 Simulator – MQTT Weather Logger

<http://www.hivemq.com/demos/websocket-client/>

Client Tools: MQTT-Explorer, MQTTX

Broker URL: broker.mqttdashboard.com

Port: 1883 (Ist der Default Port)

Subscription: wokwi-weather

➡ Save

WOKWi
World's most advanced ESP32 simulator

[Discord Community](#) [LinkedIn Group](#) [Pricing](#)

Online ESP32 Simulator

A faster way to prototype IoT projects with the ESP32 microcontrollers

Featured projects

- WiFi Scanning**
C++ ESP32 WiFi Scanning example
#include "WiFi.h"
void setup() {
 Serial.begin(115200);
 Serial.println("Scanning WiFi");
 WiFi.mode(WIFI_STA);
 Serial.println("Setup done");
}
void loop() {
 // ...
}
- NTP Clock**
C++ Learn about the ESP32 WiFi simulation in:
https://wokwi.com/projects/3209404
#include "WiFi.h"
#include "NTP.h"
#include "LiquidCrystal.h"
LiquidCrystal lcd(13, 20, 16, 2);
NTP ntp(0xDEAD, "pool.ntp.org");
DateTime localTime = ntp.getLocalTime();
#define UTC_OFFSET_DST 0
- ESP32 HTTP Server**
C++ ESP32 HTTP Server Example for Wokwi.com
https://wokwi.com/projects/3209404
To test, you need the Wokwi Cloud IDE. Check out the Wokwi website:
https://docs.wokwi.com/
Then start the simulation, and open a browser to the IP address.
In the browser, you can see the output of the server.
- Joke Machine**
C++ ESP32 HTTP Client Jokes API Example
https://wokwi.com/projects/3209404
#include "HTTPClient.h"
#include "ArduinoJson.h"
#include "Adafruit_GFX.h"
- MicroPython OLED Display**
Python From machine import Pin, I2C
import time
ESP32 Pin assignment
i2c = I2C(0, sda=Pin(21), scl=Pin(22))
oled = SSD1306(128, 64, i2c, addr=0x3C)
oled.text("Hello, Wokwi!", 10, 10)
oled.show()
- MicroPython MQTT Weather Logger**
Python MicroPython IoT Weather Station Example for Wokwi.com
To view the data:
1. Go to <https://wokwi.com>
2. Click "Connect"
3. Under Subscriptions, click "Subscribe"
4. In the Topic field, type "wokwi-weather"
Now click on the DHT22 sensor in the simulation.
You will see the temperature and humidity data being logged.

MQTT

Ausprobieren!:

Subscribe: wokwi-weather
Publish: wokwi-weather

„Hi, hier bin ich...“



Auf dem Dashboard nachsehen was angekommen ist

ESP32: WiFi Zugang

```
#include <WiFi.h>
```

```
const char* ssid = "Wokwi-GUEST";    // Dummy WLAN für IoT  
const char* password = " ... ";
```

```
WiFiClient espClient;
```

```
void setup() {  
    WiFi.begin(ssid, password);  
  
    while(WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
}
```

```
Serial.println("WiFi connected");  
Serial.println("IP address: ");  
Serial.println(WiFi.localIP());
```

ESP32 : Druck-Sensor BMP085 auslesen

```
#include <Adafruit_BMP085.h>

Adafruit_BMP085 bmp;

WiFiClient espClient;
float pressure;

void setup() {

    bmp.begin();
}

void loop() {

    pressure = bmp.readPressure();
}
```

ESP32 : MQTT initialisieren

```
#include <PubSubClient.h>
```

```
const char* mqtt_server = "broker.mqttdashboard.com";
```

```
PubSubClient client(espClient);
```

```
char msg[75];
```

```
void callback(char* topic, byte* payload, unsigned int length) { ... }
```

```
void setup() {
```

```
    client.setServer(mqtt_server, 1883); // Serveradresse, Port
```

```
    client.setCallback(callback);
```

```
    client.connect("ESP32", "myuser", "mypass")
```

```
                // Nutzernamen, Account-Name, Passwort
```

```
    client.subscribe("wokwi-weather"); // Abonniertes Topic
```

```
}
```

```
void loop() {
```

```
    client.loop();
```

```
    if (!client.connected()) { reconnect(); }
```

ESP32 : MQTT initialisieren

Diese Funktion wird ausgeführt, wenn eine Message zu einem subskribierten Topic ankommt:

```
void callback(char* topic, byte* payload, unsigned int length) { ... }
```

```
void loop() {
```

```
    client.loop();
```

```
    if (!client.connected()) { reconnect(); }
```

```
}
```

Diese MQTT Funktion prüft, ob eine Message angekommen ist und ruft gegebenenfalls die als callback() eingetragene Funktion auf.



Regelmäßig (z.B. alle 10 s oder 1 min) sollte die Verbindung zum Broker überprüft werden. reconnect() ist eine selbst zu definierende Funktion, die die Verbindung (wie in setup() {...}) wieder aufbaut.



ESP32 : MQTT publish

```
#include <stdio.h>
```

```
snprintf(msg, 75, "%d", (int)(pressure));
```

```
client.publish(" Sensors /Pressure", msg);
```

Schreibt formatiert die integer Zahl aus der Variablen pressure in den String msg (wie printf(„%d“,pressure), hier Begrenzung auf n = 75 Bytes).



Publish den String aus dem Char Array msg unter dem Topic „UliFeder/Pressure“.



Für Fließkommazahlen muss man auf dem ESP32 (oder Arduino) tricksen, weil die abgespeckten Versionen von sprintf(...) etc. keine Fließkommaformate unterstützen:

```
snprintf(msg, 75, "%d.%02d", (int)(pressure/100), (int)pressure%100);
```

```
client.publish("Sensors/Pressure", msg);
```

ESP32 : MQTT subscribe

Einkommende Messages für ein subskribiertes Topic werden an die callback(...) Funktion weitergereicht und dort verarbeitet:

```
void callback(char* topic, byte* payload, unsigned int length) {  
    if ((char)payload[0] == 't') {  
        digitalWrite(BUILTIN_LED, LOW);    // Turn the LED on  
    } else {  
        digitalWrite(BUILTIN_LED, HIGH);   // Turn the LED off  
    }  
}
```

Hier wird (einfach) nur der erste Character msg[0] im msg Array mit „t“ verglichen. Ein „t“, „true“, „tobeornottobe“ schaltete die LED ein, ein „f“, „false“ aber auch ein „xbeliebig“ die LED aus.

MQTT – Wer holt die Daten ab?

Bis hier hin:

Der Sensor hat die Messdaten aufgenommen,
sich mit dem WLAN verbunden,
sich mit dem MQTT-Broker verbunden und
die Daten an Topics geschickt. (Können wir auf unserer App sehen)

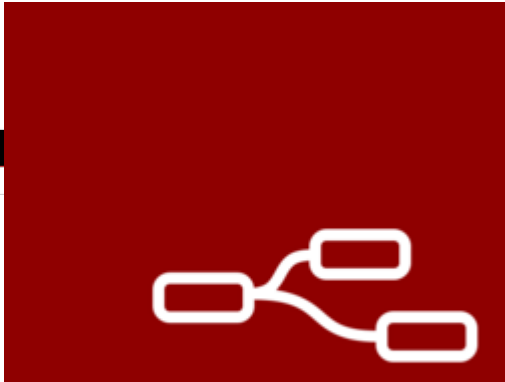
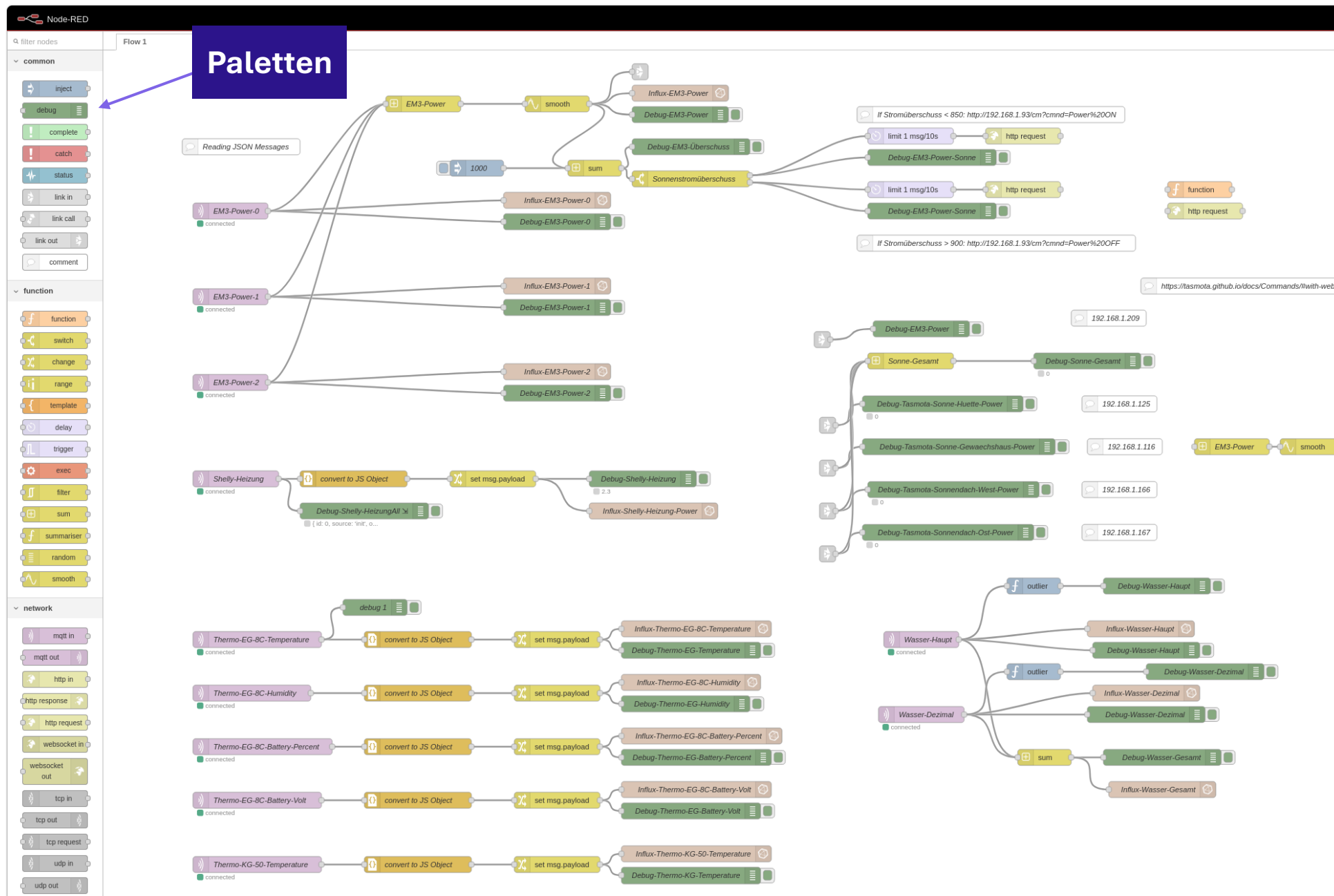
Was kann man nun mit den Daten auf dem MQTT-Broker tun?
→ MQTT kann in eigene Programmen eingebunden werden,
um mit deren Hilfe die Messdaten darzustellen.

EIN Beispiel:

→ NodeRed

Node Red

Paletten



Node-RED

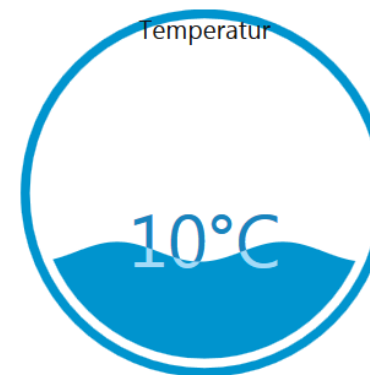
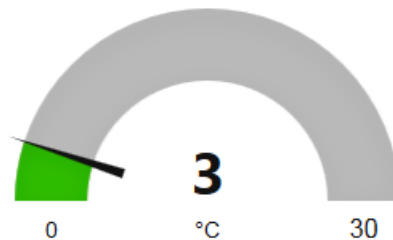
Node-Red

Flussdiagramm-basiertes Programmierwerkzeug (flow-based programming tool)

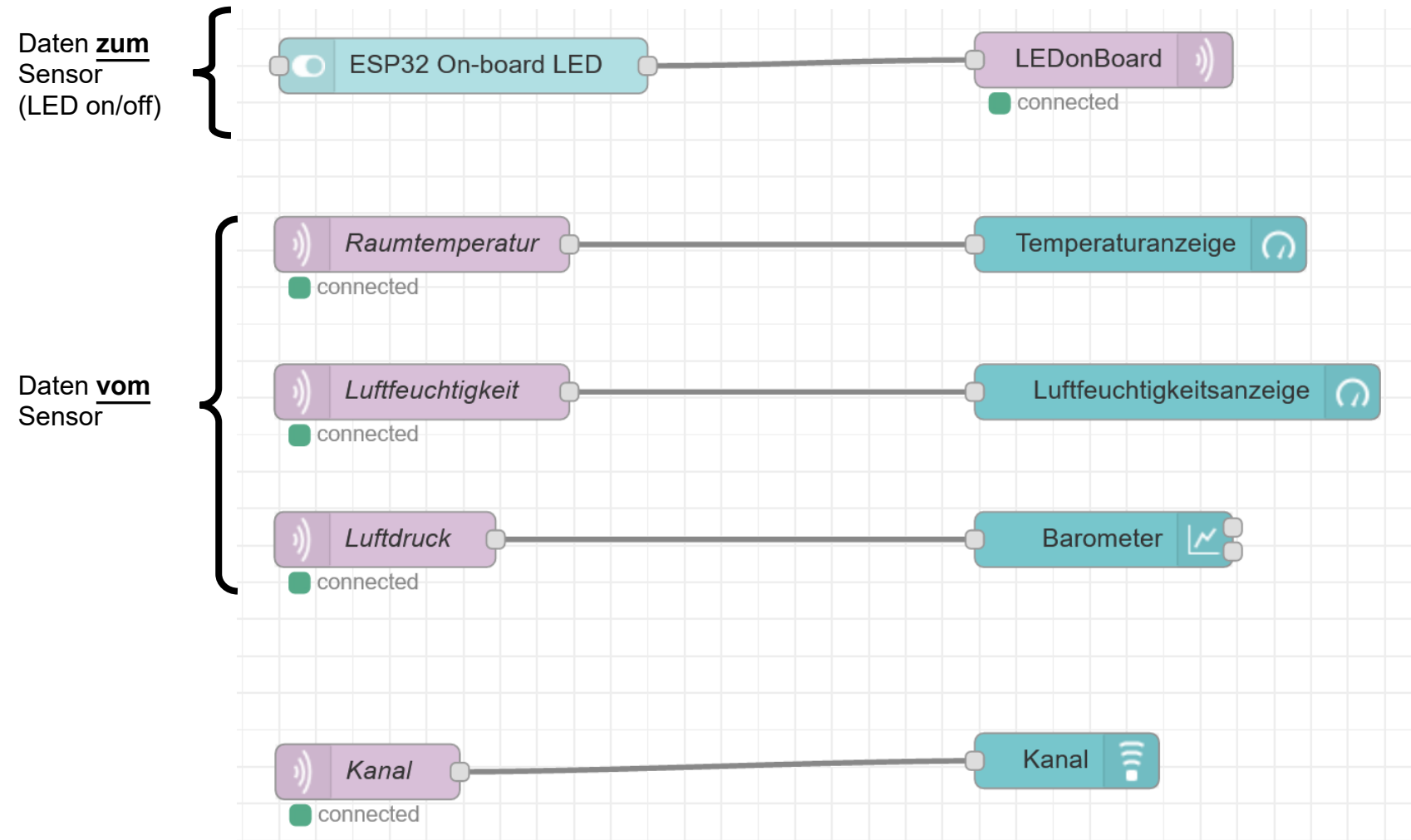


Es wird von einer Laufzeitumgebung (Programm Node-RED) ein „Dashboard“ erzeugt, das über einen Browser angezeigt werden kann:

aktuelle Temperatur in Chemnitz

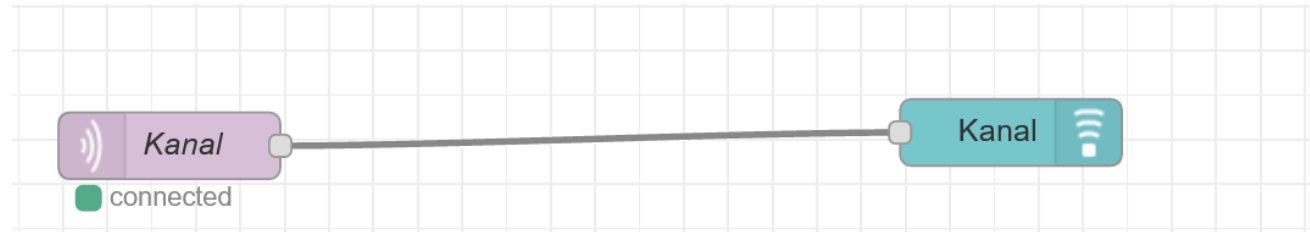


Node-Red



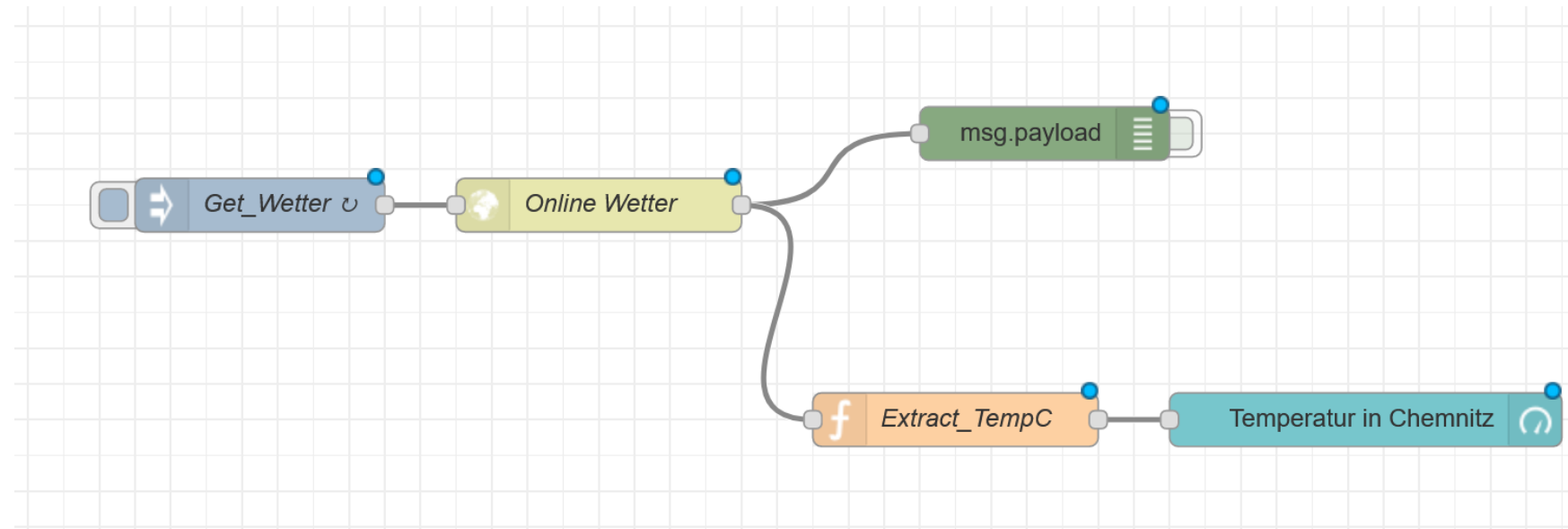
Node-Red

TTS: Text-to-speech



Alle Nachrichten an das Topic „myTopic/Kanal“ werden durch den Lautsprecher als Sprache ausgegeben.
(solange man das NodeRed-Dashboard geöffnet hat)

Node-Red



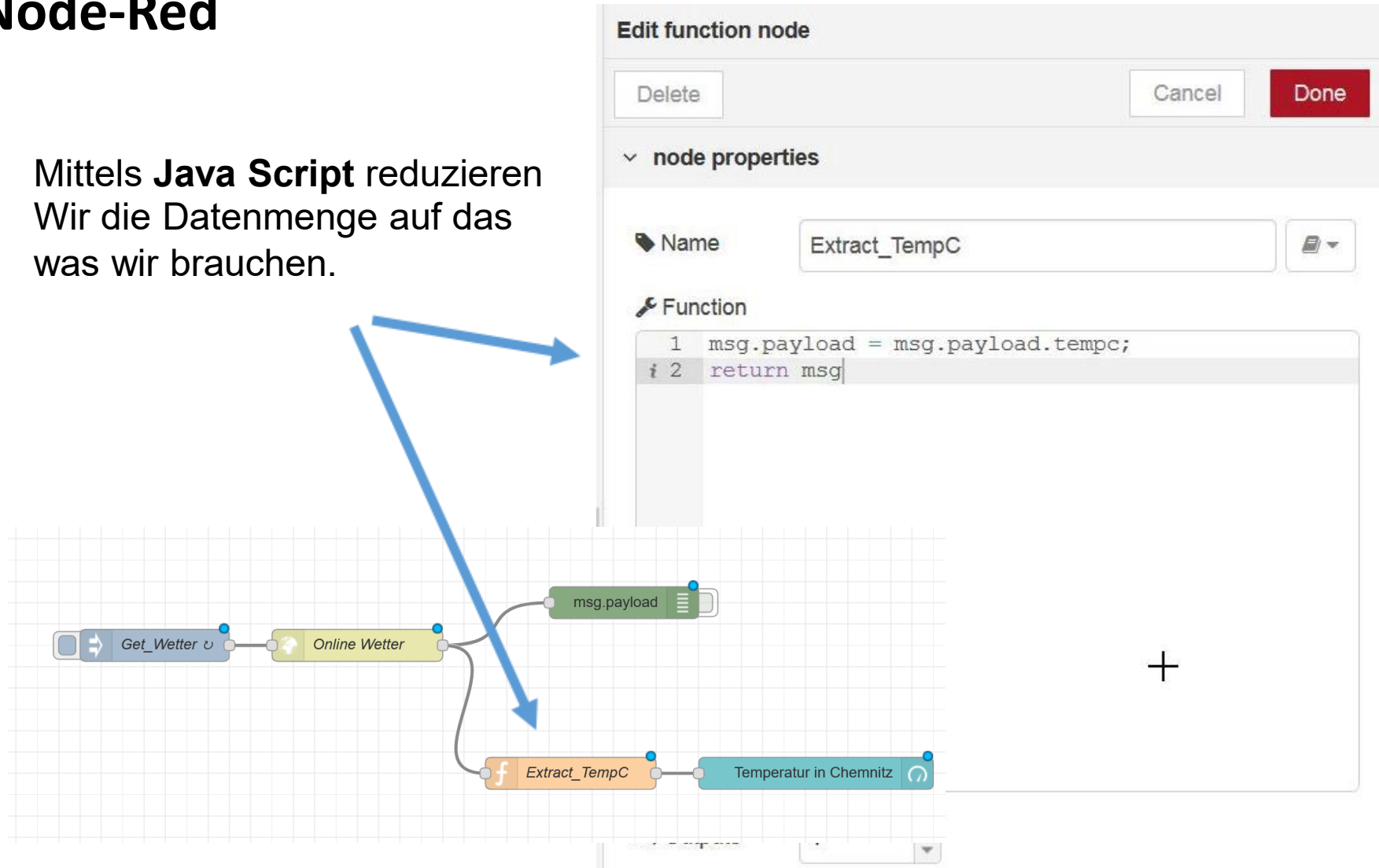
Alle 10 Sekunden aktiviert die Node „Get_Wetter“ die nachfolgenden Node „Online Wetter“, welche auf die API von OpenWeatherMap zugreift. Wir erhalten dadurch eine große Menge an Daten aus der API

Wetterdaten:

<https://home.openweathermap.org>

Node-Red

Mittels **Java Script** reduzieren
Wir die Datenmenge auf das
was wir brauchen.



Payload - msg

Was für Daten könnten wir alles bekommen?

Number

String

csv

Html

xml

wav

Video

JSON Objekt

→ häufig genutztes Format

The screenshot shows a Node-RED interface with three tabs at the top: 'info', 'debug', and 'dashboard'. The 'debug' tab is active. Below the tabs, there is a search bar with 'all nodes' and a trash icon. The main area displays a message payload for a node with ID 'a4c27bd5.32247'. The payload is a JSON object representing weather data for Chemnitz. The message is timestamped '11/01/2018, 22:39:20'. The payload object contains the following fields: 'weather' (Clouds), 'detail' (Überwiegend bewölkt), 'icon' (04n), 'tempk' (276.15), 'tempc' (3), 'temp_maxc' (3), 'temp_minc' (3), 'humidity' (93), 'maxtemp' (276.15), 'mintemp' (276.15), 'windspeed' (0.5), 'location' (Chemnitz), 'sunrise' (1515654399), 'sunset' (1515684405), 'clouds' (75), and 'description' (The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)). Below this, another message is partially visible from node '379ef5e9.e734fa', showing a payload of the number '3'. At the bottom, a third message is partially visible from node 'a4c27bd5.32247'.

```
3
11/01/2018, 22:39:20 node: a4c27bd5.32247
Wie_ist_das_Wetter : msg.payload : Object
▼ object
  weather: "Clouds"
  detail: "Überwiegend bewölkt"
  icon: "04n"
  tempk: 276.15
  tempc: 3
  temp_maxc: 3
  temp_minc: 3
  humidity: 93
  maxtemp: 276.15
  mintemp: 276.15
  windspeed: 0.5
  location: "Chemnitz"
  sunrise: 1515654399
  sunset: 1515684405
  clouds: 75
  description: "The weather in
Chemnitz at coordinates: 50.83,
12.93 is Clouds (Überwiegend
bewölkt)."
```

```
11/01/2018, 22:39:20 node: 379ef5e9.e734fa
Wie_ist_das_Wetter : msg.payload : number
3
```

```
11/01/2018, 22:39:30 node: a4c27bd5.32247
```

- **JSON – JavaScript Object Notation**

Ausgabe aus OpenWeatherMap:

```
{"weather":"Clouds","detail":"Überwiegend bewölkt","icon":"04n","tempk":276.15,"tempc":3,"temp_maxc":3,"temp_minc":3,"humidity":93,"maxtemp":276.15,"mintemp":276.15,"windspeed":0.5,"location":"Chemnitz","sunrise":1515654399,"sunset":1515684405,"clouds":75,"description":"The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."}}
```



• JSON – JavaScript Object Notation

```
{"weather":"Clouds",  
  "detail":"Überwiegend bewölkt",  
  "icon":"04n",  
  "tempk":276.15,  
  "tempc":3,  
  "temp_maxc":3,  
  "temp_minc":3,  
  "humidity":93,  
  "maxtemp":276.15,  
  "mintemp":276.15,  
  "windspeed":0.5,  
  "location":"Chemnitz",  
  "sunrise":1515654399,  
  "sunset":1515684405,  
  "clouds":75,  
  "description":"The weather in Chemnitz at  
coordinates: 50.83, 12.93 is Clouds (Überwiegend  
bewölkt)."}}
```

The screenshot shows a Node-RED interface with three tabs at the top: 'info', 'debug', and 'dashboard'. The 'debug' tab is active. Below the tabs, there is a search bar with 'all nodes' and a trash icon. The main area displays a list of nodes. The first node is selected, showing its details: '11/01/2018, 22:39:20 node: a4c27bd5.32247' and 'Wie_ist_das_Wetter : msg.payload : Object'. Below this, a collapsed 'object' is expanded, revealing a JSON payload:

```
weather: "Clouds"  
detail: "Überwiegend bewölkt"  
icon: "04n"  
tempk: 276.15  
tempc: 3  
temp_maxc: 3  
temp_minc: 3  
humidity: 93  
maxtemp: 276.15  
mintemp: 276.15  
windspeed: 0.5  
location: "Chemnitz"  
sunrise: 1515654399  
sunset: 1515684405  
clouds: 75  
description: "The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."
```

 Below the JSON, there is a second node entry: '11/01/2018, 22:39:20 node: 379ef5e9.e734fa' and 'Wie_ist_das_Wetter : msg.payload : number'. The third node entry is partially visible: '11/01/2018, 22:39:30 node: a4c27bd5.32247'.

- **JSON – JavaScript Object Notation**

... is a lightweight data-interchange format.

... completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others.

A collection of name/value pairs (objects).

An ordered list of values (arrays).

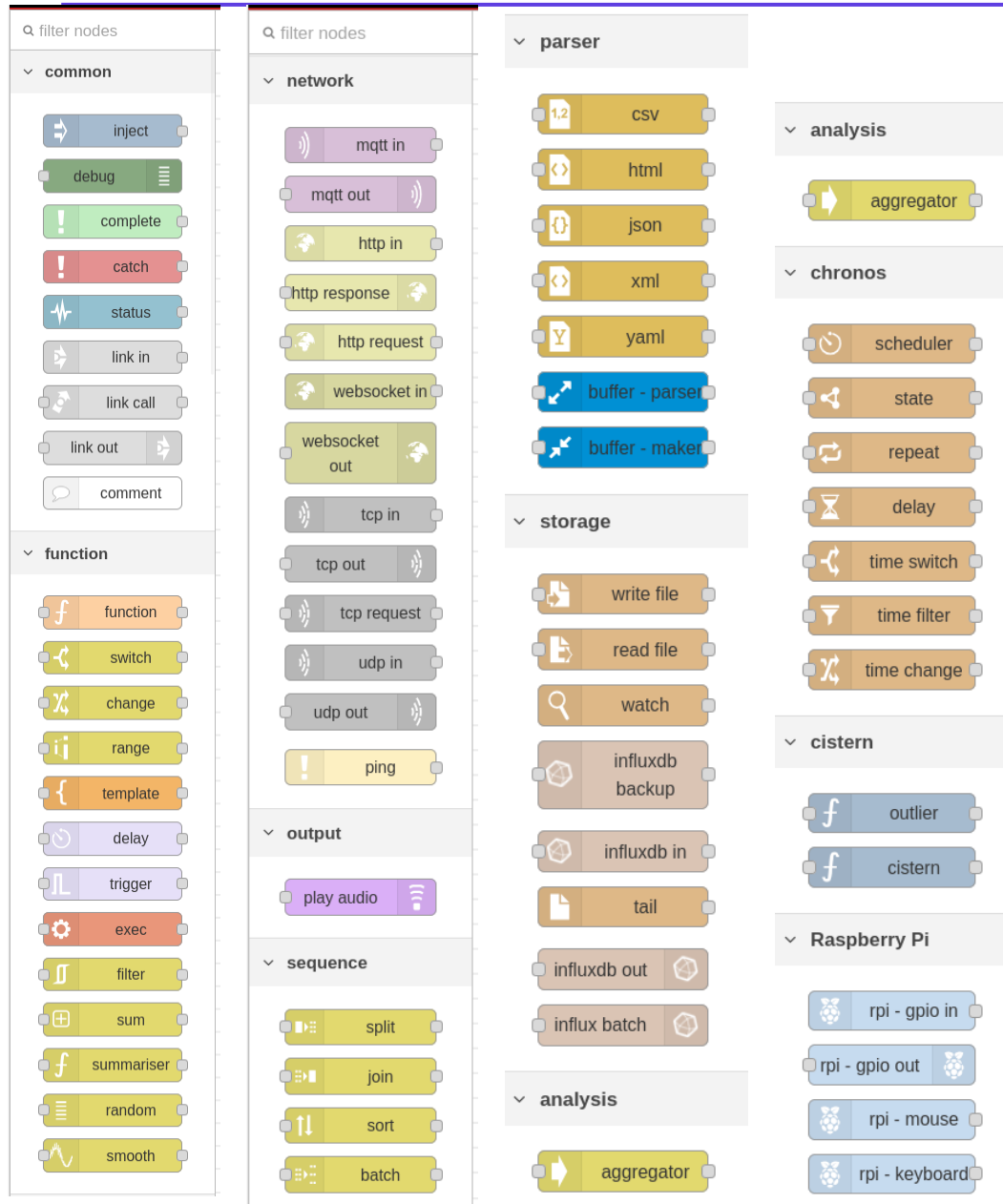
(Zitate aus <https://www.json.org/>)

Beispiel „Kreditkarte“

(aus https://de.wikipedia.org/wiki/JavaScript_Object_Notation)

```
{
  "Herausgeber": "Xema",
  "Nummer": "1234-5678-9012-3456",
  "Deckung": 2e+6,
  "Waehrung": "EURO",
  "Inhaber":
  {
    "Name": "Mustermann",
    "Vorname": "Max",
    "maennlich": true,
    "Hobbys": ["Reiten", "Golfen", "Lesen"],
    "Alter": 42,
    "Kinder": [],
    "Partner": null
  }
}
```

Node Red ist erweiterbar über Paletten



Node-RED: Nodes & Paletten

Nodes

- Bausteine für Datenverarbeitung: Input, Output, Function etc.
- Per Drag-and-Drop zu Flows verbunden.

Paletten

- Linke Seitenleiste mit Core-Nodes (ca. 40).
- Filterbar und durchsuchbar.

Erweiterbarkeit

- Tausende Community-Nodes via Palette Manager installierbar.
- Eigene Nodes entwickelbar.

Fazit

- Flexibles Low-Code-Tool für IoT-Automatisierung dank visueller Flows und hoher Erweiterbarkeit.

OpenHAB: open Home Automation Bus

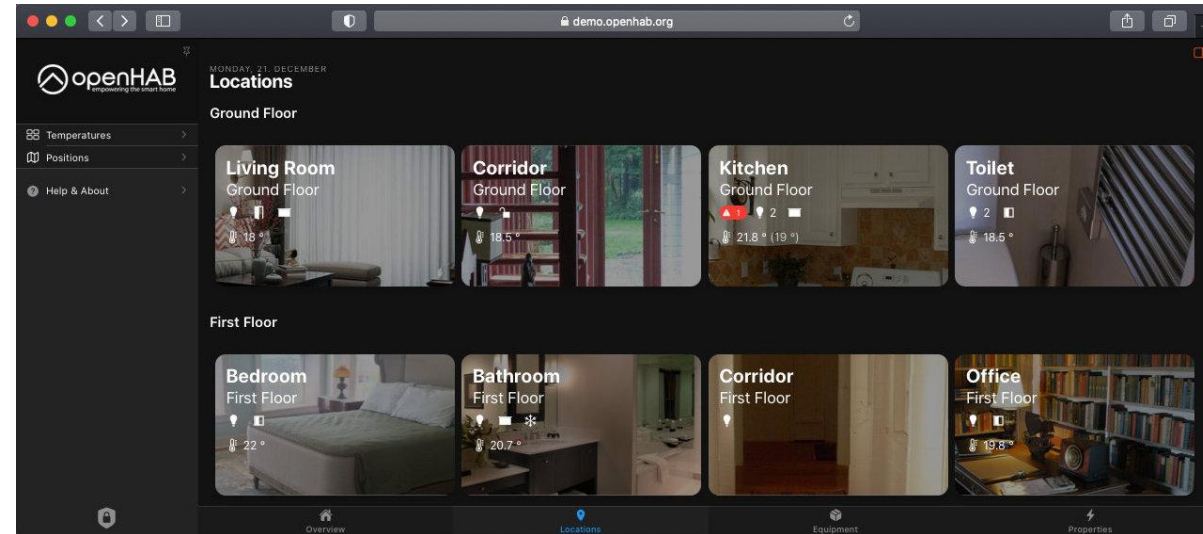
<http://www.openhab.org/>

Automatisierungssoftware für das „Smart Home“

Steuert (Licht, Heizung, Stereoanlage, Waschmaschine, Katze,...)

Bildet Strukturen ab

- Keller, Erdgeschoss, Gartenlaube,...
- Wohnzimmer, Küche, Bibliothek,...
- Heizung, Licht, Kochen, Unterhaltung,...



Speichert Zustände (Licht an, Temperatur im Wohnzimmer,...)

- Regelt (am Wochenende eine Stunde vor Sonnenaufgang das Bad heizen und die Katze füttern)

Viele „Bindings“ für Geräte von unterschiedlichen Herstellern nach unterschiedlichen Normen (KNX, ENOCEAN, HomeMatic, ...)

Visuelle Programmierung

The screenshot displays the openHAB web interface at demo.openhab.org. The left sidebar contains navigation links: Temperatures, Positions, Administration (Settings, Things, Model, Items, Pages, Rules), Scripts (selected), and Schedule. Below these are Developer Tools and Help & About. The main workspace is titled 'Simple Per-Room Climate Control' and shows a visual programming script. The script is organized into two main sections. The first section, 'to control the climate in the room with: currentRoom', includes blocks for setting 'temperatureItem' to 'Temperature', 'windowItem' to 'Window_', and 'heatingItem' to 'Heating_'. It then appends the 'currentRoom' value to each of these items. A conditional block follows: 'if windowItem == Window_FF_Office', which triggers 'to windowItem append text _Window'. The second section is an 'if' block that checks 'get item state temperatureItem < 19'. If true, it enters a 'do' block with an 'if' condition 'room has window with: roomWindow currentRoom and get item state windowItem == OPEN'. This triggers a 'log info' and 'create text with' block: 'Window open, not starting heating: currentRoom'. If the temperature condition is false, it triggers another 'log info' and 'create text with' block: 'Temperature too low, start heating: currentRoom', followed by 'send command ON to heatingItem'. The script concludes with an 'else if' block: 'room has window with: roomWindow currentRoom'. The bottom of the interface shows a 'Run Now (Ctrl-R)' button and an 'IDLE' status indicator.

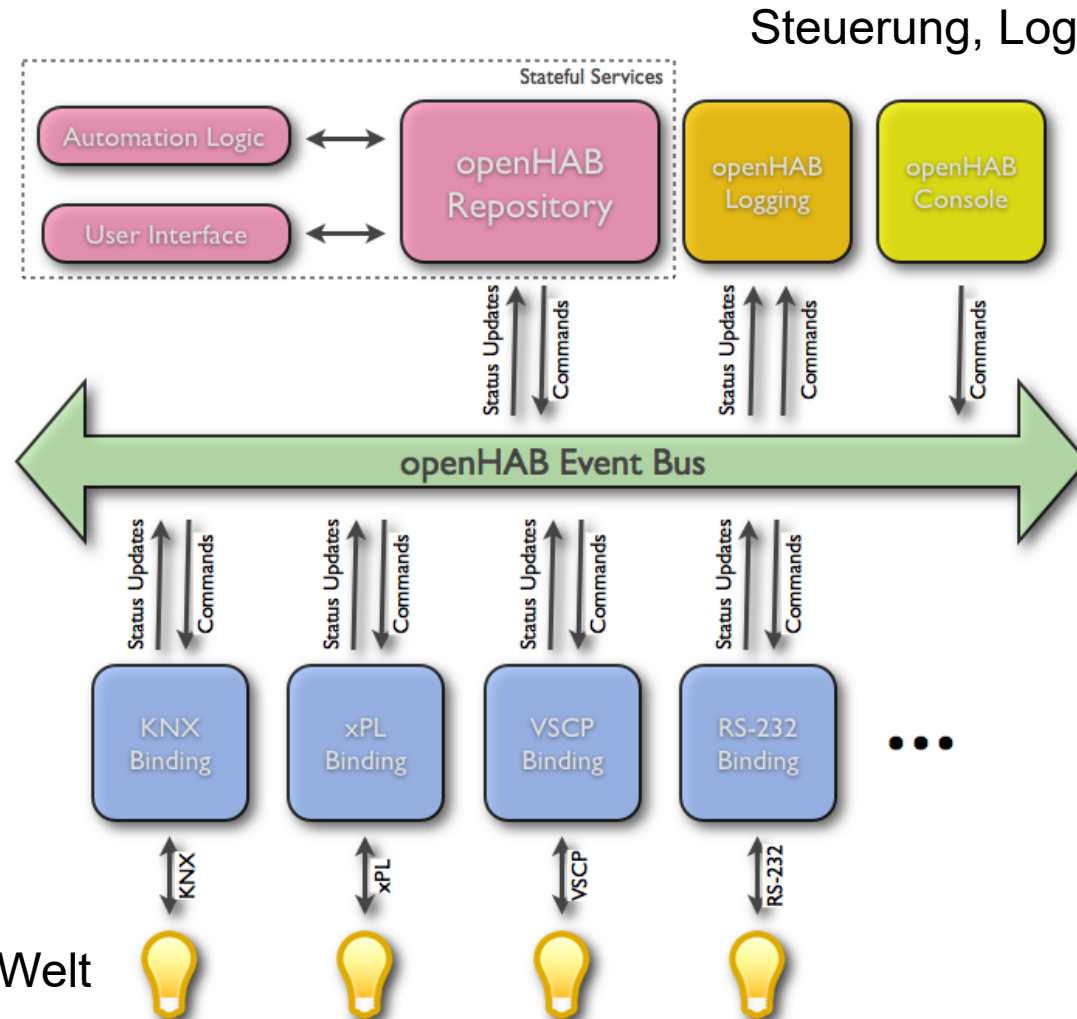
OpenHAB

Zustandsbeschreibung
und Automatisierung
(„Items“, „Rules“)

Asynchroner Ereignis-Bus

Bindings (nachlad-
und erweiterbar)

„Things“: Hardware in der echten Welt



Von Kai Kreuzer openHAB Entwickler - Eigenes [Werk](https://github.com/openhab/openhab/wiki/images/events.png)
Copyrighted free use, <https://commons.wikimedia.org/w/index.php?curid=32157828>

OpenHAB

OpenHAB gliedert sich in
Ausführender Server-Prozess: openHAB-runtime
„Benutzerfreundliche“ Konfigurationsoberfläche

Version 1.X und früher ist nur Server, die Konfiguration geht nur über Konfigurationsdateien, die „von Hand“ editiert werden müssen.

Version 2.X bietet eine Web-basierte Konfigurationsoberfläche an; trotzdem müssen Konfigurationsdateien noch editiert werden.

Die Konfiguration einer openHAB Lösung ist noch etwas für Experten, die Bedienung ist dann intuitiv über Web-Oberflächen.

OpenHAB Konfigurationsdateien

z.B. für Linux (Raspberry Pi) in den Verzeichnissen

/etc/openhab2/items

/etc/openhab2/sitemaps

/etc/openhab2/things

/etc/openhab2/rules

Text-Dateien mit den Endungen .items, .sitemap, .things, .rules

OpenHAB Konfigurationsdateien

Beispiel: Home.items Datei

Alle Objekte werden in den .items Dateien definiert (ähnlich einer Variablen- und Objekt-Deklaration)

```
Group Home          "Our Home"    <house>
Group Library       "Library"     <office>    (Home)
Group LivingRoom    "Living Room"  <sofa>      (Home)

Switch Library_Light "Light"      <light>      (Library, gLight)    {channel=""}
```

```
Number Library_Heating "Heating"  <heating>   (Library, gHeating)   {channel=""}
```

```
Number Library_Humidity "Humidity" <humidity> (Library, gHumidity)  {channel=""}
```

```
Number Library_Temperature "Temperature" <temperature> (Library, gTemperature) {channel=""}
```

```
Switch LivingRoom_Light "Light"    <light>    (LivingRoom, gLight)   {channel=""}
```

```
Number LivingRoom_Heating "Heating"  <heating>   (LivingRoom, gHeating)  {channel=""}
```

```
Number LivingRoom_Temperature "Temperature" <temperature> (LivingRoom, gTemperature) {channel=""}
```

```
Number LivingRoom_Humidity "Humidity"  <humidity> (LivingRoom, gHumidity) {channel=""}
```



```
Group:Switch:OR(ON, OFF) gLight "Light"    <light>    (Home)
```

```
Group:Number:AVG gHeating "Heating"  <heating>   (Home)
```

```
Group:Number:AVG gHumidity "Humidity"  <humidity>   (Home)
```

```
Group:Number:AVG gTemperature "Temperature" <temperature> (Home)
```

OpenHAB Konfigurationsdateien

Lichtschalter als Item:

```
Switch Library_Light      "Light"      <light>      (Library, gLight)      {channel=""}
```

Objekt ist ein Schalter (Switch) mit dem Namen („Variablennamen“) Library_Light und dem Label „Light“ (im Web-Interface sichtbare Bezeichnung), dem Icon <light>.

Das Objekt gehört zu den Gruppen (Library, gLight).

Mit dem Objekt ist die in {} eingetragene Aktion verknüpft (in diesem Fall nichts).

OpenHAB Konfigurationsdateien

Beispiel: Home.sitemap Datei beschreibt die Struktur.

```
sitemap our_home label="Our Home" {  
    Frame {  
        Group item=Library  
        Group item=LivingRoom  
    }  
    Frame {  
        Text label="Light" icon="light" {  
            Default item=Library_Light label="Library"  
            Default item=LivingRoom_Light label="Living Room"  
        }  
        Text label="Heating" icon="heating" {  
            Default item=Library_Heating label="Library"  
            Default item=LivingRoom_Heating label="Living Room"  
        }  
        Text label="Humidity" icon="humidity" {  
            Default item=Library_Humidity label="Library"  
            Default item=LivingRoom_Humidity label="Living Room"  
        }  
        Text label="Temperature" icon="temperature" {  
            Default item=Library_Temperature label="Library"  
            Default item=LivingRoom_Temperature label="Living Room"  
        }  
    }  
}
```


HomeBuilder

Please select your language

German

Home Setup Name

wohnung_admin

Floors

Floors

Draußen

Erdgeschoss

Rooms

Draußen

Lagerraum

Erdgeschoss

Gang

Schlafzimmer

Wohn-Esszimmer

Büro

Lagerraum

Badezimmer

Toilette

Objects

Lagerraum (Draußen)

Type to search or add object

Gang (Erdgeschoss)

Type to search or add object

Schlafzimmer (Erdgeschoss)

Type to search or add object

ITEMS

Copy

Group	Home	"wohnung_adeln"	<house>	
Group	OU	"Draußen"	<garden>	(Home)
Group	GF	"Erdgeschoss"	<groundfloor>	(Home)
Group	OU_StorageRoom	"Lagerraum"	<suitcase>	(Home, OU)
Group	GF_Corridor	"Gang"	<corridor>	(Home, GF)
Group	GF_Bedroom	"Schlafzimmer"	<bedroom>	(Home, GF)
Group	GF_LivingDining	"Wohn-Esszimmer"	<sofa>	(Home, GF)
Group	GF_Office	"Büro"	<office>	(Home, GF)
Group	GF_StorageRoom	"Lagerraum"	<suitcase>	(Home, GF)
Group	GF_Bathroom	"Badezimmer"	<bath>	(Home, GF)
Group	GF_Toilet	"Toilette"	<toilet>	(Home, GF)

SITEMAP

DIGITALE WELT

www.digitalewelt.at

OpenHAB Konfigurationsdateien

Beispiel: myHouse.items

Hier wird ein MQTT Binding eingesetzt, um mit das EPS8266 Board mit seinen Sensoren einzubinden.

```
Group      My      "EPS8266"      <office>

Switch     SchalterRoteLED      "RoteLED"      <light> (Feder)
           { mqtt=">[mosquitto:LEDonBoard:command:ON:true],>[mosquitto:LEDonBoard:command:OFF:false]" }

Number     My_Humidity      "Humidity [%.2f rel. percent]"      <humidity>      (myHouse)
           {mqtt="<[mosquitto:Sensors/Humidity:state:default]"}

Number     My_Temperature      "Temperature [%.1f °C]"      <temperature>      (myHouse)
           {mqtt="<[mosquitto:Sensors/Temperature:state:default]"}

Number     My_Pressure      "Luftdruck [%.0f Pa]"      <pressure>      (myHouse)
           {mqtt="<[mosquitto:Sensors/Pressure:state:default]"}
```

OpenHAB Rules

Dateien `RegelName_oder_so.rules` in `/etc/openhab2/rules`

Deklaration von Variablen (gelten innerhalb einer Regel-Datei):

```
var counter = 0           // a variable with an initial value. Note that the variable type is automatically inferred
val msg = "This is a message" // a read-only value, again the type is automatically inferred
```

Regeln:

```
rule "<RULE_NAME>"
when
    <TRIGGER_CONDITION> [or <TRIGGER_CONDITION2> [or ...]]
then
    <SCRIPT_BLOCK>
end
```

OpenHAB Rules

Ereignisse können Regeln auslösen (triggern):

Item <item> received command [<command>]

Item <item> received update [<state>]

Item <item> changed [from <state>] [to <state>]

Von Timer ausgelöste Regeln:

Time is midnight

Time is noon

Time cron "<cron expression>"

... und weitere Möglichkeiten

OpenHAB Rules

Beispiel Dimmer:

```
rule "Dimmen Light"
```

```
when
```

```
    ITEM DimmndLight received command
```

```
then
```

```
    var Number percent = 0
```

```
    if(DimmndLight.state instanceof DecimalType) percent = DimmndLight.state as DecimalType
```

```
    if(receivedCommand == INCREASE) percent = percent + 5
```

```
    if(receivedCommand == DECREASE) percent = percent - 5
```

```
    if(percent < 0) percent = 0
```

```
    if(percent > 100) percent = 100
```

```
    postUpdate(DimmndLight, percent);
```

```
End
```

OpenHAB Rules

Beispiel: „Channel“ des „Astro“ Binding löst eine Regeln aus:

```
rule "Start wake up light on sunrise"
when
    Channel "astro:sun:home:rise#event" triggered START
then
    ...
End
```

Online Documents

MQTT

<https://www.heise.de/developer/artikel/MQTT-Protokoll-fuer-das-Internet-der-Dinge-2168152.html>

Node-Red

<https://nodered.org/>

JSON

<https://www.json.org/>

<https://en.wikipedia.org/wiki/JSON>

https://de.wikipedia.org/wiki/JavaScript_Object_Notation

openHAB

<https://docs.openhab.org/>

<https://docs.openhab.org/concepts/index.html>

<https://docs.openhab.org/tutorials/beginner/index.html>

Literaturempfehlung für den tieferen Einstieg: Walter Trojan, Das MQTT-Praxisbuch – mit ESP8266 und Node-RED (elektor Verlag, 2017, 1. Auflage)